

Algoritma: Bir problemin a5adılması zmn izlenen yol, yntem

İkili → Binary

ABG /

50

tabanlar

10 (decimal) → 0-9

16 (Hex) → 0-9 A-F

2 (Binary) → 0-1

8 (Octal) → 0-7

Onluk → İkili

$(45)_{10} = (?)_2$

$(45)_{10} = (101101)_2$

32 → 1

16 → 0

8 → 1

4 → 1

2 → 0

1 → 1

En yksek anlamlı

$45 - 32 = 13$

En dksek anlamlı

$13 - 8 = 5$

$5 - 4 = 1$

bit  
bit

İkili → Onluk

$(101101)_2 = (?)_{10}$

00101101 → 1 byte

$(101101)_2 = 1 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 + 0 \times 2^4 + 1 \times 2^5 = 1 + 4 + 8 + 32 = 45_{10}$

Adreslendirm en kucuk bilgi: 8 bit → 1 byte

bit → 0 veya 1 deeri alabilen en kucuk bilgi.

Adreslere 8'in katı olarak zunda. 8 bit → 16 bit → 32 bit → 64 bit

$(11111111)_2$

$128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 \rightarrow 255$

EYA

EDA

7 6 5 4 3 2 1 0

-128 → 127

İşaret biti  
(sign bit)

0: +

1: -

7 6 5 4 3 2 1 0

0 0 1 0 1 1 0 1

+ büyüklük kısmı



128

↑ 10000000 ↑

01111111 → 1'e tamleyen  
(1's complement)

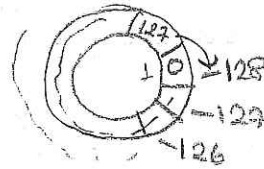
10000000

→ 2'ye tamleyen  
(2's complement)

ifaret  
biti

↓  
-128

(NEGATIVE)



10000000

01111111

+ 1

10000000 ⇒ -128

-22

+22 00010110

11101001 1'e tamleyen

+ 1

11101010

2'ye tamleyen

ifaret  
biti

11101010

00010101

+ 1

(00010110)<sub>2</sub> → (22)<sub>10</sub>

-22

←  
7k 12  
kadar  
sana  
tamleyen

(00010110)

↓

11101010

(01111111)<sub>2</sub> = (127)<sub>10</sub>

↓↓↓↓↓↓

10000001

ifaret  
biti

Floating Point Binary

2.5 × 10<sup>-3</sup>

mantissa

exponent

sign

-3.154 × 10<sup>5</sup>

3 × 10<sup>0</sup> 1 × 10<sup>-1</sup> + 5 × 10<sup>-2</sup> + 4 × 10<sup>-3</sup>

mantissa

(11.1011)<sub>2</sub> × 2<sup>3</sup> = (3.6875 × 8)<sub>10</sub> = (29.5)<sub>10</sub>

3 1 ×  $\frac{1}{2}$  + 0 ×  $\frac{1}{4}$  + 1 ×  $\frac{1}{8}$  + 1 ×  $\frac{1}{16}$

$\frac{8+2+1}{16} = \frac{11}{16}$

İkili

Onluk Gösterim

Onluk Değer

|        |      |   |        |
|--------|------|---|--------|
| • 1    | 1/2  | → | • 5    |
| • 01   | 1/4  | → | • 25   |
| • 001  | 1/8  | → | • 125  |
| • 0001 | 1/16 | → | • 0625 |

• 101

↓

• 5 + 125

Onluk → İkili Dönüştürme

$$(2.625)_{10} = (---)_2$$

$$(2.625)_{10} = (10.101)_2$$

↓

$$0.625 \times 2 = 1.25 \quad \underline{1}$$

$$0.25 \times 2 = 0.5 \quad \underline{0}$$

$$0.5 \times 2 = 1.0 \quad \underline{1}$$

$$(0.40625)_{10} = (---)_2$$

$$0.40625 \times 2 = 0.8125 \rightarrow 0$$

$$0.8125 \times 2 = 1.625 \rightarrow 1$$

$$0.625 \times 2 = 1.25 \rightarrow 1$$

$$0.25 \times 2 = 0.5 \rightarrow 0$$

$$0.5 \times 2 = 1.0 \rightarrow 1$$

$$(0.01101)_2$$

$$(-4.75)_{10} = (---)_2$$

$$0.75 \times 2 = 1.5 \rightarrow 1$$

$$0.5 \times 2 = 1 \rightarrow 1$$

$$(-100.11)_2$$

$$(1.7)_{10} = (---)_2$$

$$0.7 \times 2 = 1.4 \rightarrow 1$$

$$0.4 \times 2 = 0.8 \rightarrow 0$$

$$0.8 \times 2 = 1.6 \rightarrow 1$$

$$0.6 \times 2 = 1.2 \rightarrow 1$$

$$0.2 \times 2 = 0.4 \rightarrow 0$$

$$0.4 \times 2 = 0.8 \quad \text{Devir}$$

$$(1.10110)_2$$

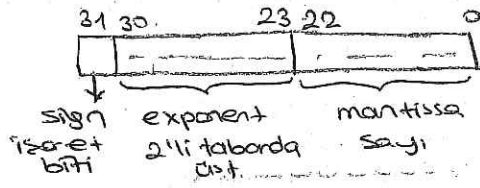


## IEEE 754 standard

Single precision: 32 bit (short real)

double precision: 64 bit (long real)

32 bit :



Örnek düzeydeki bir sayının 32 bit düzeyinde (yaklaşık 7.22) karşılığı

- 1) Sayı 32 bit düzeyine getirilir.
- 2) Noktanın solunda tek bir bit kalacak şekilde (tek bütüne bir bitin değeri 1 olacak şekilde) sayı normalize edilir.
- 3) Noktadan sonrası mantissa bölümüne yazılır. Üste (exponent) offset eklenerek elde edilen sayı 32 bit düzeyine getirilir ve exponent bölümüne yazılır.

belli  
anada  
ilavesi  
32 bit

İşaret biti yazılır.

32 bit kesirli sayılarda offset değeri 127'dir.

$$(3.14)_{10} = (---)_2$$

$$(11.0010001111010111...)_2$$

↓  
11.

$$0.14 \times 2 = 0.28 \quad 0$$

$$0.28 \times 2 = 0.56 \quad 0$$

$$0.56 \times 2 = 1.12 \quad 1$$

$$0.12 \times 2 = 0.24 \quad 0$$

$$0.24 \times 2 = 0.48 \quad 0$$

$$0.48 \times 2 = 0.96 \quad 0$$

$$0.96 \times 2 = 1.92 \quad 1$$

$$0.92 \times 2 = 1.84 \quad 1$$

$$0.84 \times 2 = 1.68 \quad 1$$

$$0.68 \times 2 = 1.36 \quad 1$$

$$0.36 \times 2 = 0.72 \quad 0$$

$$0.72 \times 2 = 1.44 \quad 1$$

$$0.44 \times 2 = 0.88 \quad 0$$

$$0.88 \times 2 = 1.76 \quad 1$$

$$0.76 \times 2 = 1.52 \quad 1$$

$$0.52 \times 2 = 1.04 \quad 1$$

$$0.04 \quad \}$$

$$0.04 \quad \}$$

$$\frac{36}{2} = 18$$

$$\frac{32}{2} = 16$$

$$\frac{84}{2} = 42$$

$$\frac{68}{2} = 34$$

$$\frac{88}{2} = 44$$

$$\frac{96}{2} = 48$$

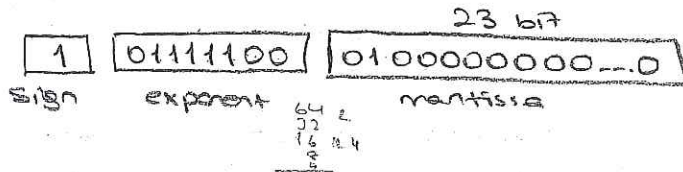
$$11.0010001111010111$$



↓                      ↓                      ↓  
 sign bit    exponent    mantissa

base: 127

1)  $-0.00101 = -1.01 \times 2^{-3}$   
 $\rightarrow$   
 exponent =  $127 - 3 = 124$



floating point  
 estimate  
 sogenannte  
 (daß wir nicht 100%)

$1.01.010 \rightarrow 1.01010 \times 2^2$   
 $\leftarrow$   
 $127 + 2 = 129$

$(1 \ 10000010 \ 1111011000000000...0)_2 = (?)_{10}$

$-1.111011 \times 2^3$

$1.11101$   
 $\leftarrow$

$128 + 2 = 130$

$(1111,1011)_2 = 15,6875$

$\frac{1}{2} + \frac{1}{8} + \frac{1}{16} = \frac{11}{16}$

floating point

$(1e100 - 1e100) + 1.0 \approx 1.0$

$(1e100 + 1.0) - 1e100 \approx 0.0$

$(x+y)+z \neq x+(y+z)$

$(x \times y) \times z \neq x \times (y \times z)$

0.1 wird hier zweimal von oben heruntergezogen

$0.1 \times 2 = 0.2$   
 $0.2 \times 2 = 0.4$   
 $0.4 \times 2 = 0.8$   
 $0.8 \times 2 = 1.6$   
 $0.6 \times 2 = 1.2$   
 $\rightarrow 0.2$

0-9

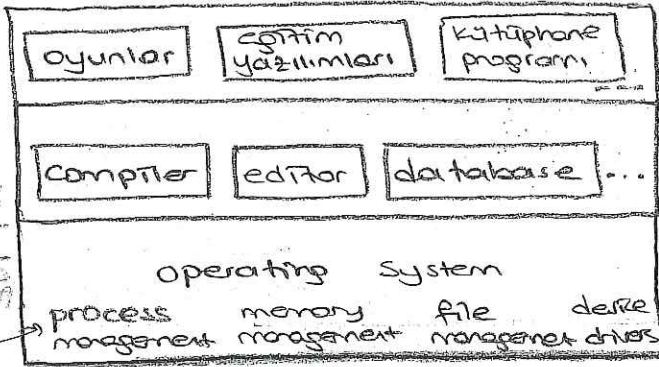
A-F

$23F \rightarrow 2 \times 16^2 + 3 \times 16^1 + 15 \times 16^0$



## Bilgisayar Sistemi

2 parçadan oluşur. software-hardware



uygulama yazılımları (application software)

sistem yazılımları (system software)

compiler: yazdığımız programların bilgisayarın anlayabileceği hale getiren yazıdır.

database: belli bir hiyerarşiye göre bilgilerin saklanabilmesi, gerektiğinde

etip kullanılabilmesini sağlar. (sistem yazılımı)

paket pro.  
son kullanıcının kullanmasını sağlar

işletim sist.: programların doğru ve birlikte kullanılabilmesini sağlayan

process management: site belli esnada, gelmiş yapılacak işin

saat aynı anda yapılmıyormuş gibi görülecek yere getirir.

(mouse-begitme, etc) aslında aynı anda değil. yanlışlık durumu da var

memory management: belirlenmiş bir bellek var.

yapılan işler orada tutulur. kullanımı kontrol eder.

neresi boş, orada tut. kullanması: bu kadar site belleğin 3/4'ü kullanılabiliyor, orada arkaya 1 mb yavaş imişlik olur olur yer biter mümkün olduğunca yer yavaş yer bulup.

eskiden böyle değildi ama virtual memory.

immişlik de bile 10 mb'lık yer kullanabili

götürür.

bellekte dizeğin bir şekilde saklanabilmesi

file management: dosyaların yapılara göre uygun şekilde saklanabilmesi

ve getirilebilmesi. directory- ağaca yapısı

device drivers: donanımlar printer, mouse, video kartı

bütün bilg. donanımla bağlantısını ve kullanılabilmesini sağlayan programlar.

printer kullanılabilir, bilgisayarı gönderme, haberleşebilir

donanım-yazılım operating system

|          |                  |
|----------|------------------|
| HARDWARE | machine language |
|          | microprogramming |
|          | physical devices |

machine code  $\rightarrow$  native dille qewirli. Ove dilerde alisen konvertor.  
native dille dogrudan nakest Bilen birli tegrada oia elligke konvertor  
their. (doonma dogrudan kicredibilmek dogayon)

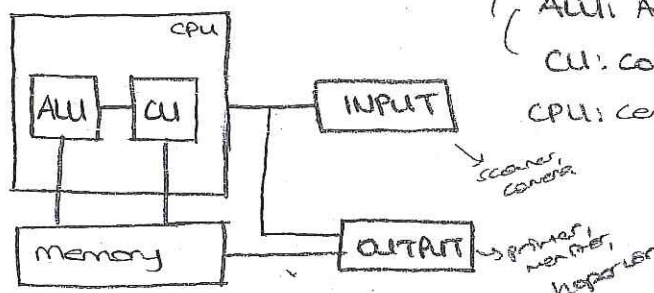
neuroprogramming: collige bene etiam neuro-komutatoras aliquid

bu komiteler yaptırılması sağlanır. Listelenen

Registra masinari, 21je tiparjenja aluminij mprskadista mbrpropozicije

physical devices : program documents, circuitry, papers.

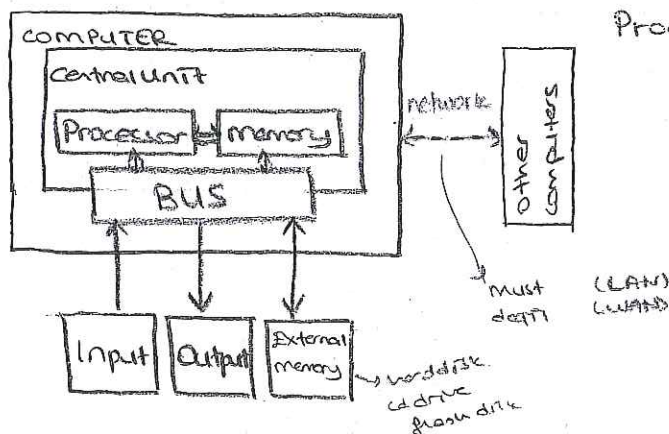
Von-Neumann Mimarisi



## ALU Arithmetic-Logic Unit

C.U: Control Unit (bills in control) 18.10

CPU: Central Processing unit



Data Bus - Address Bus

BUS: <sup>strimmer</sup> cheater, brinker, arcade light, trigger, atom, haberdashery, sp. play



2 tır  
bellek

## Bilgi Saklama

Volatile : main memory RAM

Random Access memory

non-Volatile : external (auxiliary)

RAM, sequential

### Volatile (ucunak)

bilgi kapadığı anda piden bilgileri kaybolan bellek (RAM)

RAM'in içindeki gür. (1-4 GB) Saniye de (10 GB)

bilg. azlık olduğu süre geçmi: olan bellek

### non-volatile

bilg. kapana bile pinge bilgileri tutar (external derke)

CD, DVD, hard disk, teyp, floppy disk

erism 2 tır

volatile'da RAM Random Access memory

değide erişilebilir.  
CD, DVD

non-volatile RAM, sequential

Y small erism

teyp, kaset

erism RAM sarmak, geçim

örce suayla, Reklam, Challenge erime RAM

## Saklayabilir bilgi

$10^{-12}$  : piko

$10^{-9}$  : nano

$10^{-6}$  : mikro

$10^{-3}$  : mili

$10^3$  : kilo

$10^6$  : mega

$10^9$  : giga

$10^{12}$  : tera ✓

1024 byte (8 bit)

1024 byte

ALGORITHM A Bigliayada koribosha sonlar ushbu amaliy jallistirilgan

4  
El-Horezmi

Bilgi'de taşınım problem aynı gibi

1st generation  $\rightarrow$  make diff

2nd generation  $\rightarrow$  Assembly  $\rightarrow$  Low Level Prog. lang.

makrini ibaradina laqah oia eller LBSE makriner nuy  
konutlar dora onlatilar peetide LBSE Assesment del 2001

konutlar daha sağlıklıdır zira

`mov ax, bx`  
`add bx, 5`

assembler  $\rightarrow$  machine code

assembler aracılığıyla makineye talimatlar verilebilir  
makine dilinde yazılmış olan bir veride bilgilerin aradıkları  
şekle getirilmesi

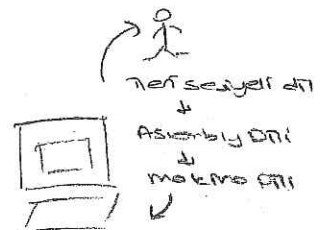
3rd generation  $\rightarrow$  High Level Programming Language

Pascal, C, Fortran

Mobile - 1000

A 5th page.  
you took better than  
last time. The only thing  
important.

READ A



Doküman geliştirilerek, yapılan görüşmeler, bilimsel araştırmalar, medya ve diğer gelişmeler.

4th generation

SQL  
query language

bakıcılık, hastane, mükterik, sporci viki otomasyon gibi yitilmek  
Tatadiginda diler kullanimk potansiyer.

```
SELECT MISTERI FROM MISTERILISTE WHERE  
MISTERI_NO = 3157
```

5th generation → Prolog, Lisp gibi (uygun teknolojiler)  
→ Garbel dil (Visual C++, Visual Basic)

⁂ Dabgoi dille, kumina diti? sibi ana Prolog, Upe gili yopay tala amogan

Kulturen aller (dogm. all. Tiere, Mensch, gibt Konstante für alle Nationen)

Aggregat einer  $\mathbb{Z}$ -Modul  $C$ ,  $C \neq 0$ , ist ein  $\mathbb{Z}$ -Modul  $B$  mit  $B \cong C$ .

yapılan eğitim etrafında dönüyor. (görsel bir bilgi var)

Şişir yavaşca var, müzik enter 1 bozluu doldurma ile

1'de fette gibi bürçölde, grafik bir terehim lenden ve saphores

## Object Oriented Programming Lang.

→ C++  
Java  
C++

neşre değeri prog. dili = yapının gereken tür obje her biri ve objeler kullanılıyor.



Compiler = Her source dili, makine diline çeviren program  
(Derleyici)

Hedef 1) Yazdığınız programın hata analizi yapar

1) İmla hataları (Syntax Analysis)

for i=1 to 10 do  
↓  
i:=1

i:=5) yanlışdır.

2) Semantik analiz (Semantic Analysis)

mantıksal hataları veya yanlışları  
bulur

~~a+b:=c~~

c:=a+b

Program koda çevirir, çalıştırma kodu verir.

2) Arka işlemleri yapıyor. çalıştırma kodu ve programın izi  
verir.



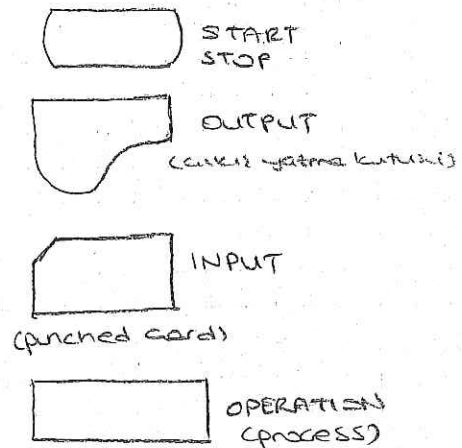
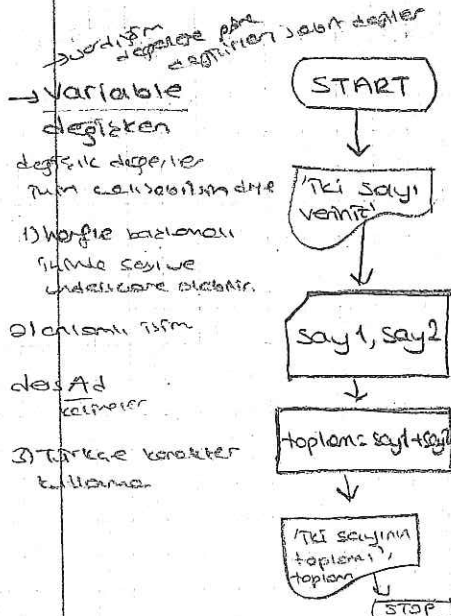
1. Akış Diyagramı + Analiz  
(Flow chart)

2. Yarı kodlar  
(Pseudo code)

1. GİRİŞ: İki sayı...

2. İşlem: verilen iki sayı arasında toplama işlemi

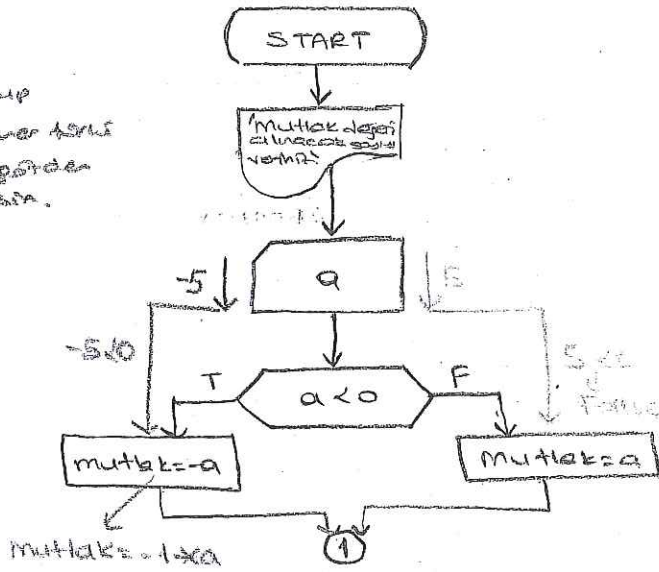
3. ÇIKIŞ: İki sayının toplamı



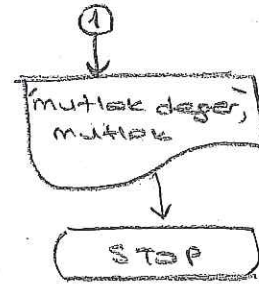
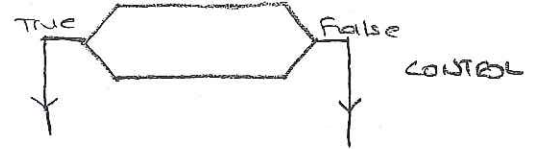


1. Giriş: Sayı
2. İşlem: Sayı 0'den küçükse pozitif yapılmalı
3. Çıkış: Sayının mutlak değeri

Sayıya kurup  
olabildiğince verileri  
planlayıp gözden  
geçirmeliyiz.



bu işlemin sonucunu  
kontrol etmek için  
= 2 } kontrol yapalım



### Analiz

$a = 5$   
 $5 < 0$  ?  
 F → mutlak = 5

$a = -5$   
 $-5 < 0$  ?  
 T → mutlak = 5

$a = 0$   
 $0 < 0$  ?  
 F → mutlak = 0

→ girilen 1 sayı  
alınacak olan

ÖRNEK 1) Ki sayı veriliyor, küçük olan sayının değerini ekrana yazdıran algoritmayı tasarlayınız

2) a, b, c katsayıları verilen 2. dereceden  $ax^2 + bx + c$  gibi bir denklemin köklerinin genel olduğu söyleniyor. Bu kökleri bulan algoritmayı tasarlayınız.  $x_1, x_2$  nedir?

terminal  
↓  
servera bağlan  
kaldırık yok

terminal → command line  
command prompt

dosya işleme, dosya sistemi yapısı

> ls = terminali açtıktan sonra home / 1106601

> cp  
> mv  
> pwd  
> cd  
> mkdir  
> touch  
> rm  
> rmdir  
> chmod  
> chown  
> man  
> clear

> ls dirdeki dosya ve klasörler dır

→ adları sadece  
klasör

gizli dosyaları →  
göster

> ls -la

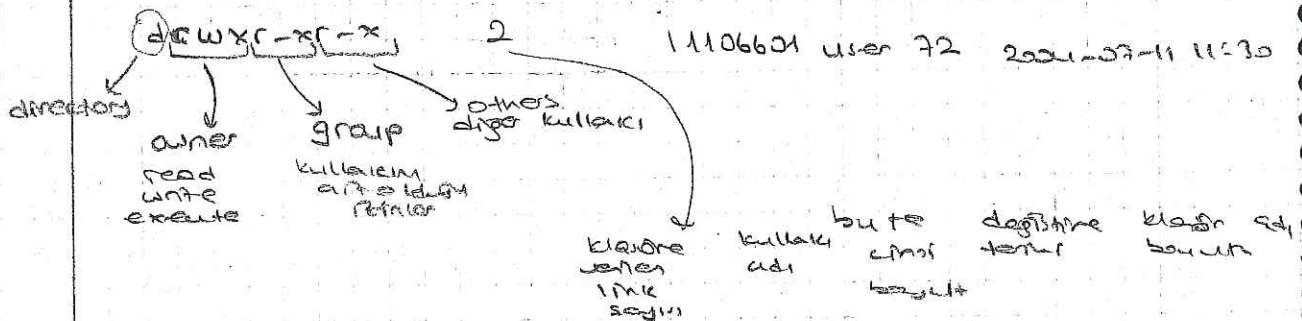
• karakter ile başlayan klasör isimler

> ls -l

Bir klasörün veya dosyanın ayrıntılı bilgilerini  
verir.

bu  
sıra da

> ls -la



daya, kle, or  
diti

home  
bbylab 1 - k u s s e r  
bbylab 2

parametresini aynı şekilde  
"cult 4" gibi  
TK yede donya kalem

```
print working directory
C:\Users\jagm\Documents\adit\adit\adit
/home/11106601
```

Scd bloglab1 → closer rule pattern

1/home/11106601/ bzglab4

get  
done

have 1 turn

Director  
2141 Turner

Drink die azeel

✓ cd need

$$\sim /neel > cd \rightarrow$$

yes  
dada  
yours

2rm test.txt parametre olarak dönecek adı  
(511erim ve gar deneşim)

(directly after the war)  
after the war was over

change mode

bkr dnyk wpa  
 kuzash  
 rin baklon  
 dptirchik.

215 - 1

$I$   $\frac{I}{R}$   $\frac{I}{R}$   $\frac{I}{R}$

dağıtım  
olduğu için her bir direnç için  
direnç - voltaj oranı  
eşit olmalıdır.

$$110 \mid 110 \mid 110$$

2 like ditten  
6 6 6

$$y_{15} - 1 \text{ test. } 4 \times 4$$

-chmod 644



> chown <yenisekip > dosyaadi >

Change owner

dosyanın sahibini değiştirir

root

kullanıcıları, dosya yetkileri, grupları, izinleri

parametre olarak alır

> man ls

↓  
manuel

kullanıcılar

ne yapar, parametreler alır

"q" ile çıkar

> clear

www.ce.yildiz.edu.tr

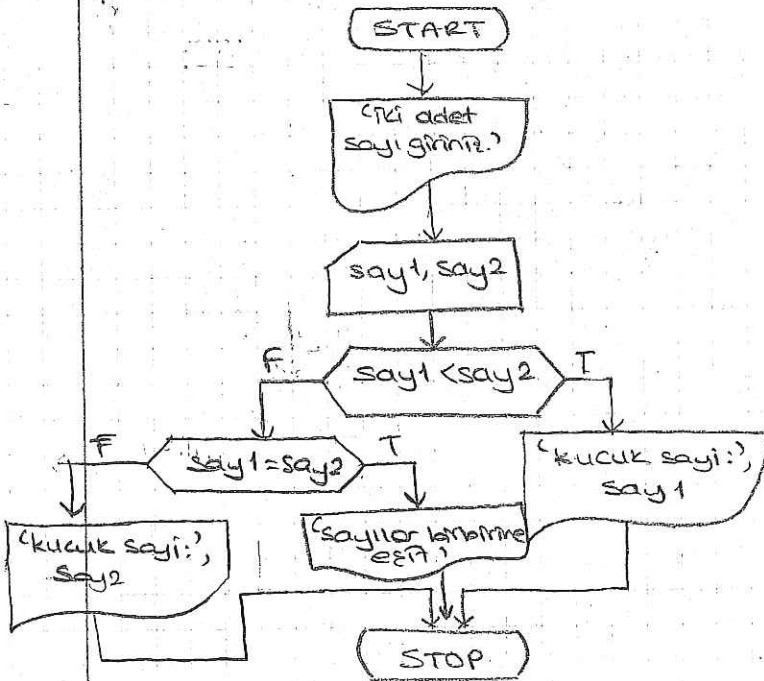
documents

SW Lab related

— 0 —

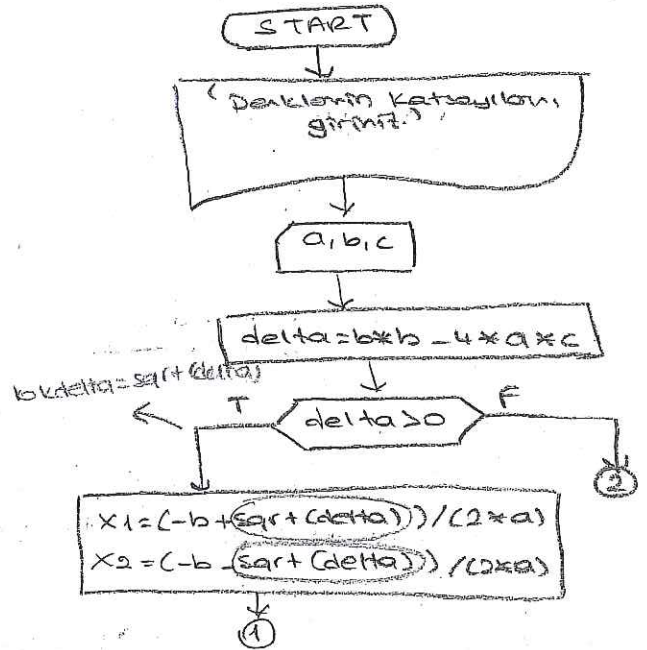
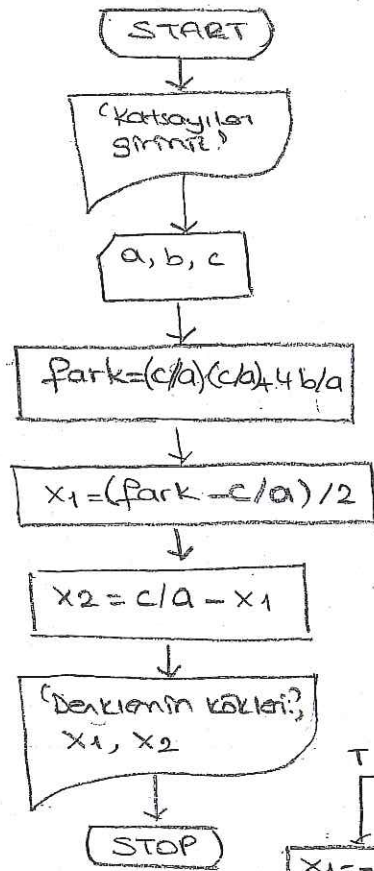
dev

1) İki sayı veriliyor, küçük olan sayının değerini ekrana yazdıran prog.



2) a, b, c katsayıları verilen 2 dereceden  $ax^2+bx+c=0$  gibi bir denklemin köklerinin gerçel olduğu bitmiyor. Bu kökleri veren algoritma?

Bazı da  
sqrt(delta)  
square root

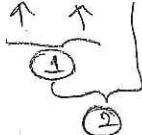


En yavaş olanı, durum göre çıkış diyagramını çiz. Aynı olmamı bekliyorsak TRUE koluna onu yaz.

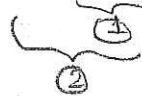
① Anlatılır  
② Kısık

İşlem Önceliği

$$a \leftarrow b * b + c$$



$$a \leftarrow b + b * c$$



$$x_1 = \frac{-b + \sqrt{\text{delta}}}{2 * a}$$

( ) olmasa 2'ye göre  
a neyse.

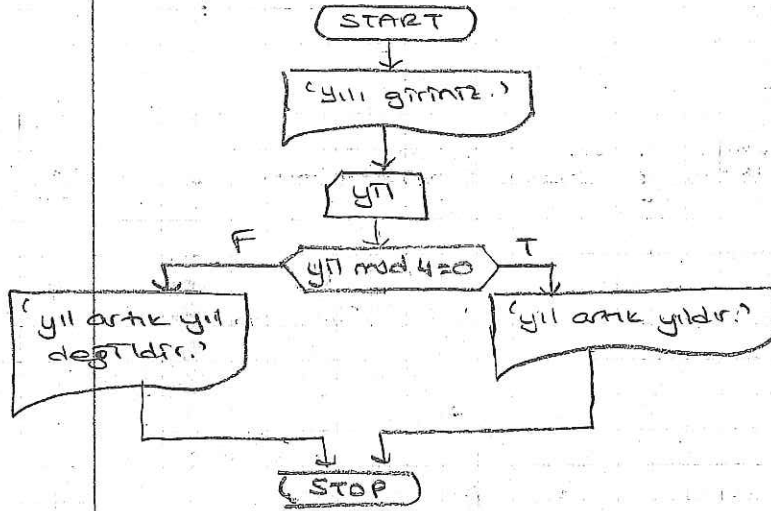
1. ( )

2. \* /

3. + -

÷  
\*  
/  
mod

Artık yıl 4'e tam bölünen yıl olduğuna göre bir yılın artık yıl olup olmadığını bulmak yazdığınız algoritmayı tasarlayınız



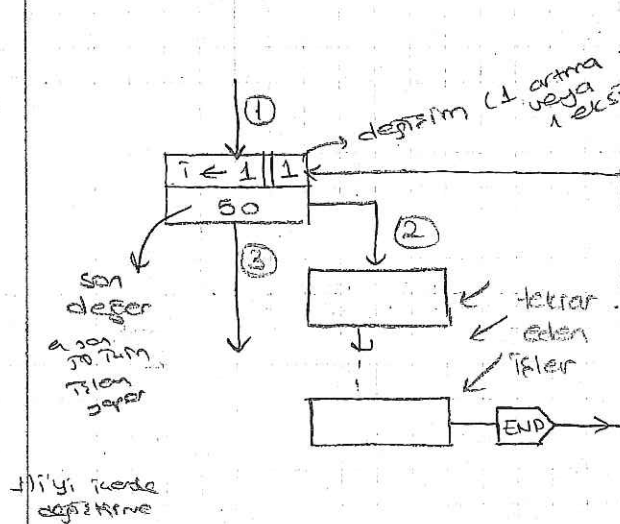
Yılın 4'e tam bölünmesi  
↓  
değilse  
↓  
4'e bölünmesi  
↓  
4'e bölünmesi

1. Giriş
2. İşlem
3. Çıkış

### Döngü Kullanımı (LOOP)

1. Adım sayısı belli olan
2. Adım sayısı belli olmayan

index → i  
↓  
k  
Görüntüden



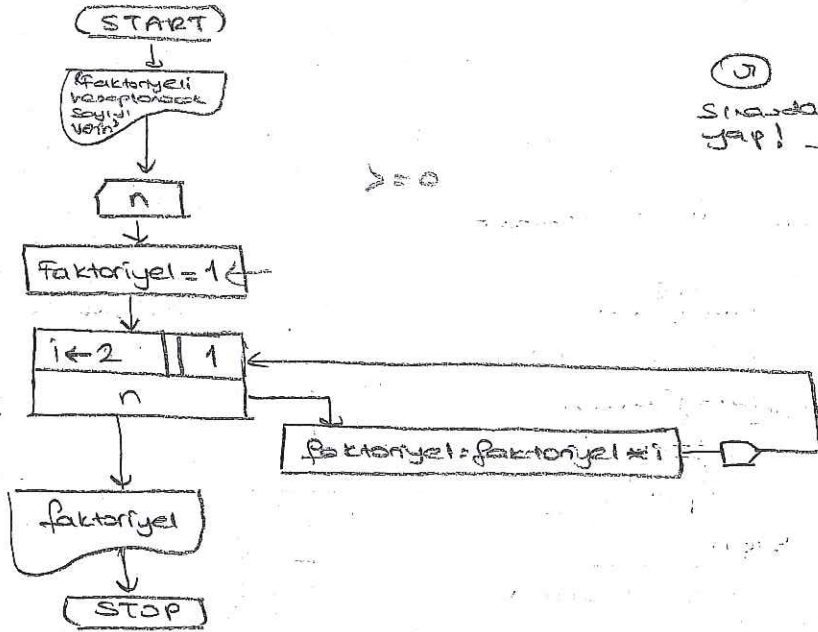
→ başlangıç  
kutu  
son göster.

for i:=1 to 50 do  
for i:=50 downto 1 do

İliyi i'nde  
değiştirme



Verilen bir sayının faktöriyel koşulunu veraplayan algoritmayı tasarla



$7 \rightarrow 7! = 1 \times 2 \times 3 \times 4 \times 5 \times 6 \times 7$   
 $4 \rightarrow 4! = 1 \times 2 \times 3 \times 4$

# Analit

$\frac{n}{3}$ 

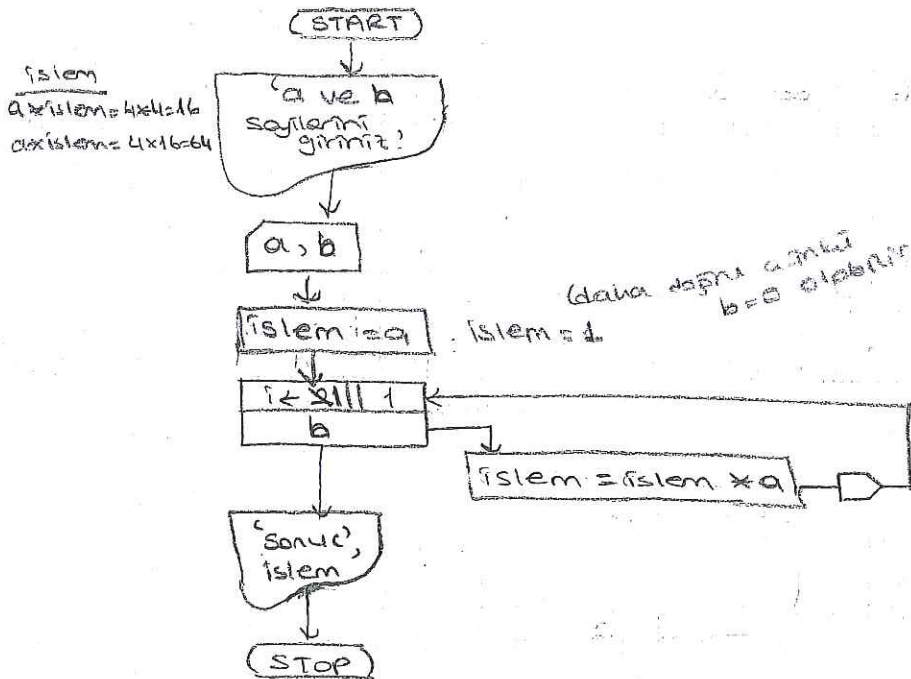
| <u>faktoriel</u> | <u>i</u> |
|------------------|----------|
| 1                | 2        |
| 2                | 3        |
| 6                | 4        |
| 1                | 2        |

7/21/2011 10:11 AM

25

$$a^4 \rightarrow a \times \underbrace{a \times a \times a}_3$$

Bit  $a$  sayısının  $n$ . dereceden kuvveti  $a$  sayısının  $(n-1)$  defa kendisiyle çarpılmasıdır. Buna göre verilen  $a$  ve  $n$  sayıları  $RM$   $a$  sayısının  $n$ . dereceden kuvvetini hesaplayan algoritmayı tasarlayınız.


$$\begin{array}{r} 1 \\ 2 \\ 3 \\ 4 \end{array}$$

system  
 $4 \times \text{system} = 4 \times 4 = 16$   
 $4 \times \text{system} = 4 \times 16 = 64$

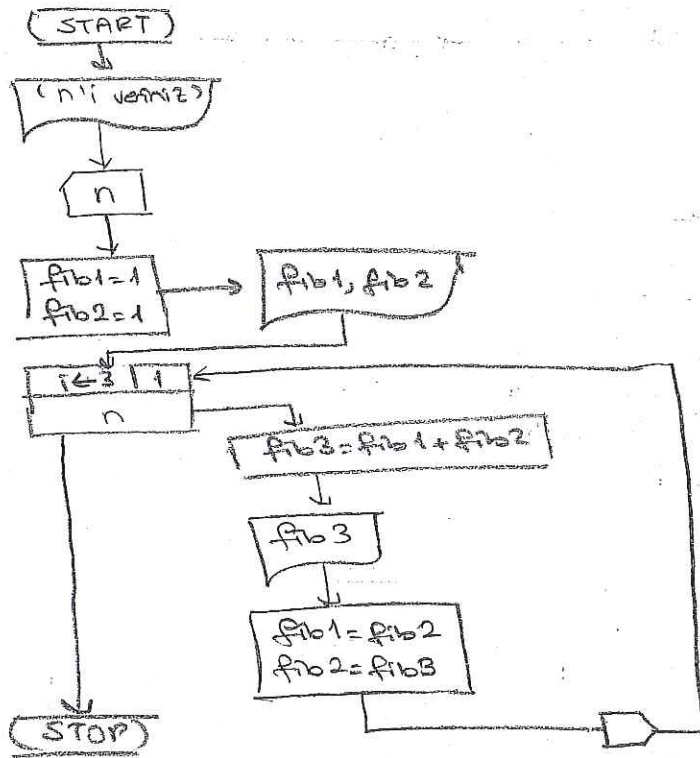
(data dari a dan b = 0 elektron)

ang life  
desises deperes  
telt.  
  
kullomaton Bank  
oapm deperes  
in deperes  
set telt  
kulla yson  
adana byn

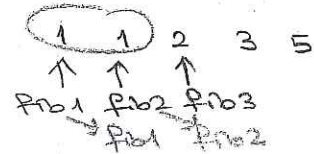
temp = 9 ✓

~~TSIARC =  $Q \times Q$~~   
 ~~$Q = Q \times Q$~~   
 Lokations Kartens  
 (größen)  
 $2 \times 2 = 4$   
 $4 \times 4 = 16$





2.1 den de baslangic



N=7

| i | fib1 | fib2 | fib3 | fibn |
|---|------|------|------|------|
| 3 | 1    | 1    | 2    | 2    |
| 4 | 2    | 3    | 3    | 3    |
| 5 | 3    | 5    | 5    | 5    |
| 6 | 5    | 8    | 8    |      |

Program fibonacci (input,output);

var

n, i, fib1, fib2, fib3 : integer;

begin

writeln ('n sayisini veriniz.');

readln (n);

fib1:=1;

fib2:=1;

writeln (fib1, fib2);

for i:=3 to n do

begin

fib3:=fib1+fib2;

writeln (fib3);

fib1:=fib2;

fib2:=fib3;

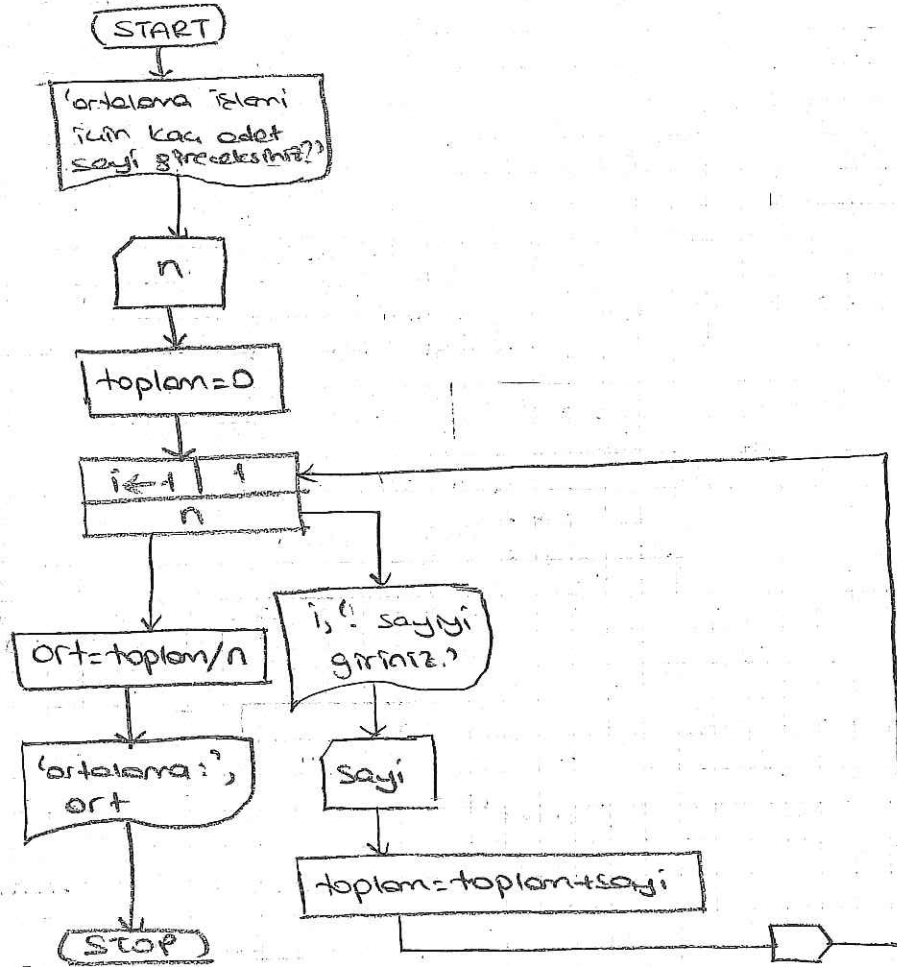
end

end.



HW

Disorolar girilen n adet sayiyi toplayarak ortalama suu yildran  
algoritmayi tasarlayip programni yaziniz



Program ortalama (input, output);

var

toplam, n, i, sayi : integer;

ort : float;

begin

writeln('ortalama islemi iuin kac adet sayi girecektiniz?');

readln(n);

toplam:=0;

for i:=1 to n do

begin

writeln(i, ' sayiyi giriniz.');

readln(sayi);

toplam = toplam + sayi

end;

ort = toplam / n;

writeln('ortalama:', ort)

end

| n | i | sayi | toplam | ort    |
|---|---|------|--------|--------|
| 4 | 1 | 2    | 2      |        |
|   | 2 | 4    | 6      |        |
|   | 3 | 6    | 12     |        |
|   | 4 | 8    | 20     |        |
|   | 5 |      |        | 20/4=5 |

Dizi  $\rightarrow$  Array

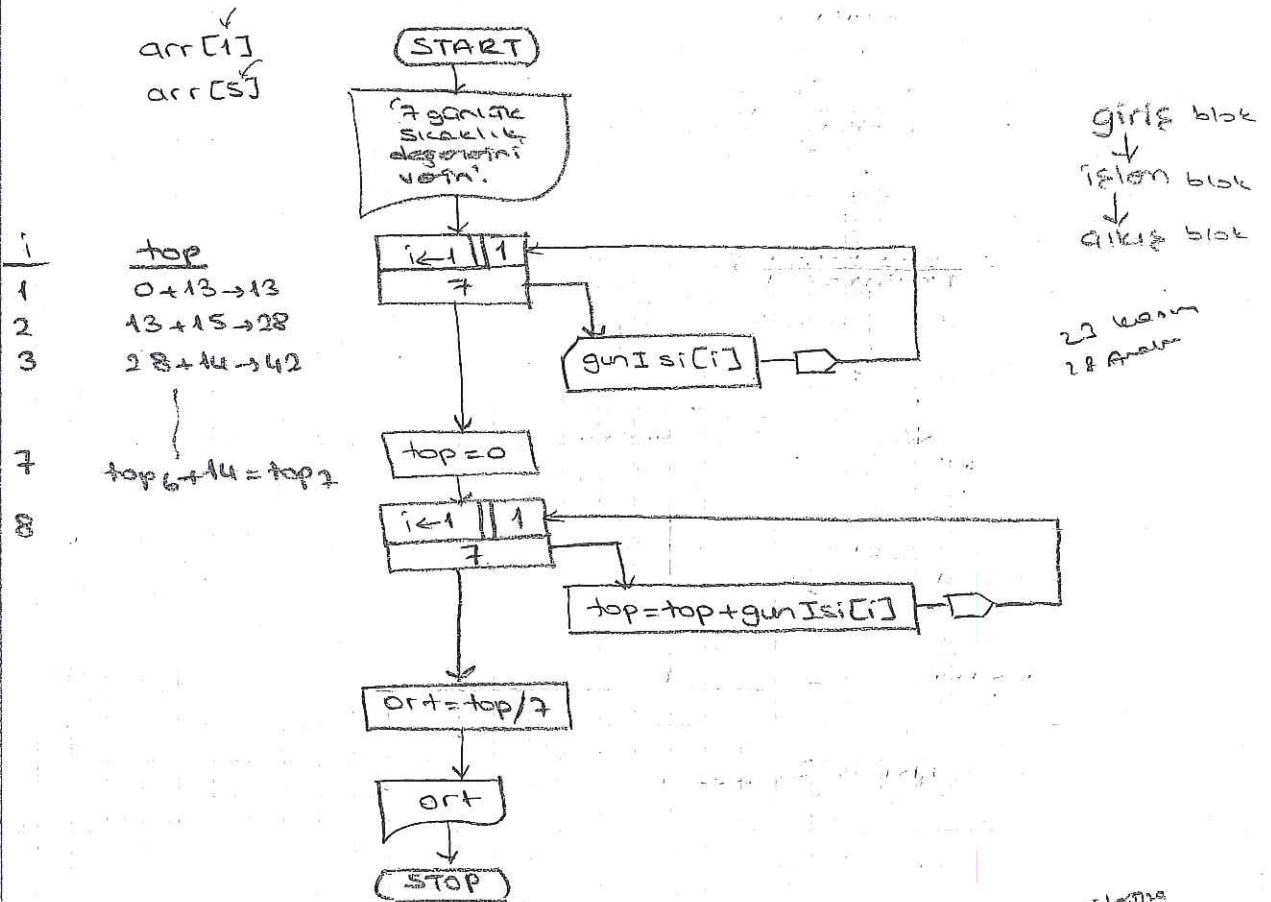
Aynı anda 7m kullanan, otok değışikli bingil.

var

arr: array [1..10] of integer;

indix      base logic      bit's indix      index  
 ↓      ↓      ↓  
 1 2 3      10  
 arr [ 9 8                ]

Bir bilgisayar haftalık sıcaklık değeri veriyor. Bu bilgisayar 0 haftan  
 itibaren sıcaklık değeri hesaplayan algoritmayı tasarayalım.



```
program ortIsi (input, output);
```

vor

i, top, ort : integer;

gunIsi : array [1..7] of integer;

begin

for  $i := 1$  to 7 do

begin

Werte in  $(i, c)$  - genau 15 dazwischen verstreut.

```
readh (goinisi [1])
```

End:

```
top := 0;
```

for  $i := 1$  to 7 do

top := top + gunIsLit;

GIRI

15LEM

Shirleye yeni belirdiyse  
matrikl alır.

0.10000  
2.0000  
0.0000  
0.0000



```

ort := top div 7;
writeln('7 günlük ın deęerler ın
ortalama: ', ort)
end.

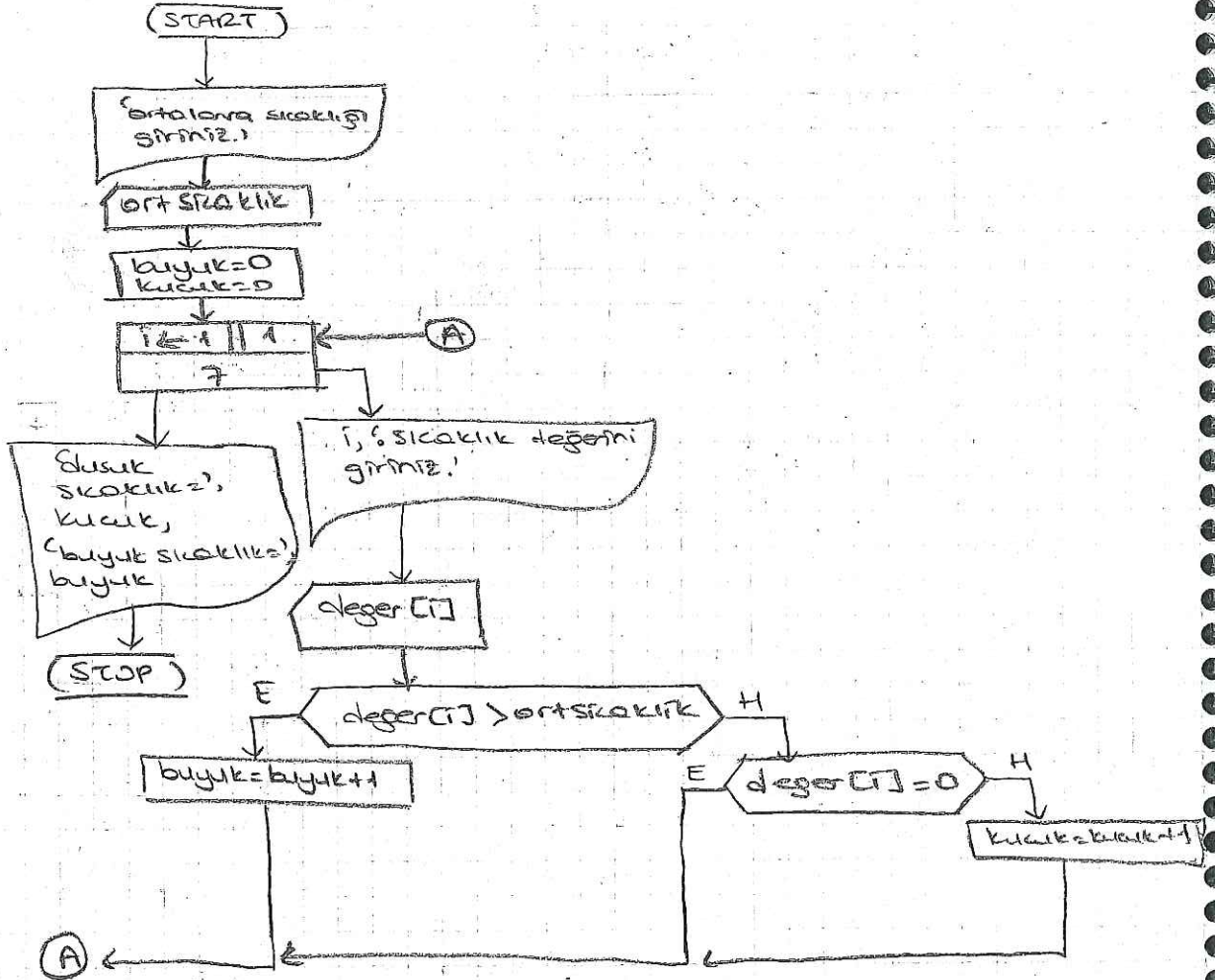
```

tam sağı bölme: div  
kesirli bölme: /

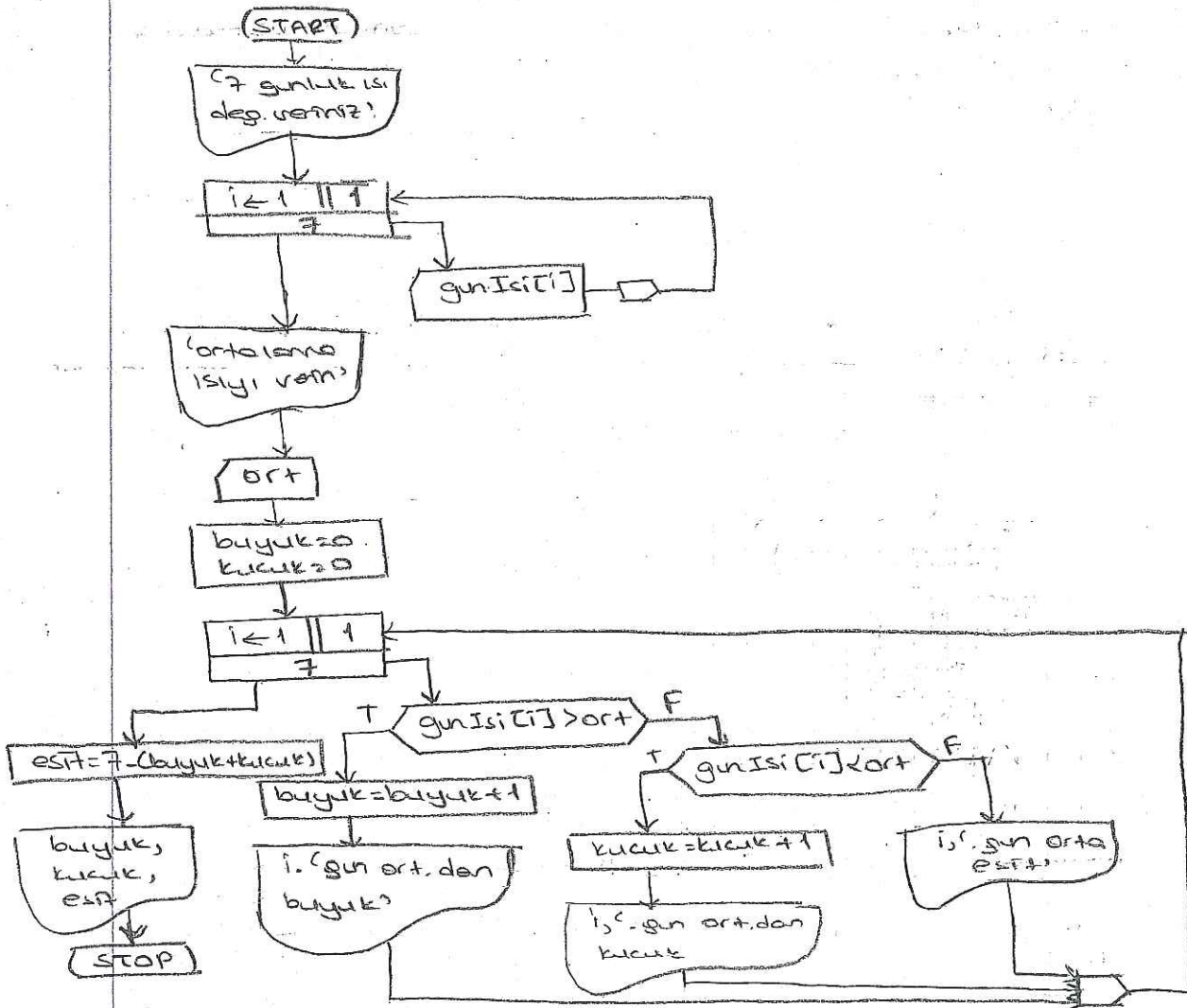
1'de parantezli  
ama 2 al,  
dörtüde kutnu

/\* — — — \*/

1X Bir bölgede 7 günlük sıcaklık deęerleri ve ortalama sıcaklık deęeri veriliyor. kaç günün ortalama sıcaklıktan büyük, kaç günün küçük olduğunu hesaplayan algoritmayı tasarlayınız







gunIsi

| 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|----|----|----|----|----|----|----|
| 20 | 21 | 22 | 18 | 15 | 23 | 17 |

↓  
gunIsi[i]

ort = 19 //

| i | gunIsi[i] | buyuk | kucuk |
|---|-----------|-------|-------|
| 1 | 20        | 0     | 1     |
| 2 | 21        | 1     | 0     |
| 3 | 22        | 2     | 0     |
| 4 | 18        |       | 1     |
| 5 | 15        |       | 2     |
| 6 | 23        | 4     |       |
| 7 | 17        |       | 3     |

$$esit = 7 - (4 + 3) = 0$$

buyuk → 4

kucuk → 3

esit → 0

> → Buyuk  
 >= → Buyuk esit  
 < → kucuk  
 <= → kucuk esit  
 = → esit  
 if a = b

başlıklar  
if  
anlamına

Bir sıraydaki 20 öğrencinin numarası veriliyor. Ayrıca bir X sayısı veriliyor. X sayısı bu öğrencilerden birinin numarası olduğuna göre, kaçınca öğrencinin numarası olduğunu bulan algoritmayı tasarlayınız.

9 7 13 4  
(13)

değer  
↓  
deneme  
konusu  
ver.

n elemanlı bir dizinin elemanları pozitif ve negatif sayılardan oluşmaktadır. Bu sayıların mutlak değerini hesaplayarak yat bir diziye yerleştiren algoritmayı tasarlayınız.

→ a: [-9 | 4 | 0 | -5]  
→ mut: [9 | 4 | 0 | 5]

LAB

Cihan

20 MB

/home/11106601/

⇒ du -sm.

disk usage

dosyaların ne kadar yer tuttuğu

rm -f

dosyaları siler

top

İşletim sistemi kullanımları

(process)

her process için PID

ID'i

ne kadar işlem yapmış

herhangi bir programın ne kadar çalıştığını

gösterir. Her process için  
kullanılan işlemci miktarını

kill -9 process-id

process-id process'i durdurmak için

ps -ef

Sistemindeki bütün processleri gösterir.

o anki durumu gösterir.

Sisteminde hangi PID olan processler çalışıyor

ps -ef | grep 11106601

alt gr. >1  
<1

man man

man kullanırken ilgili data  
manual pages

man du

man printf

C programı



space ne ↓

9 ile çıkış

quit

allık  
değer  
yerine  
gösterir

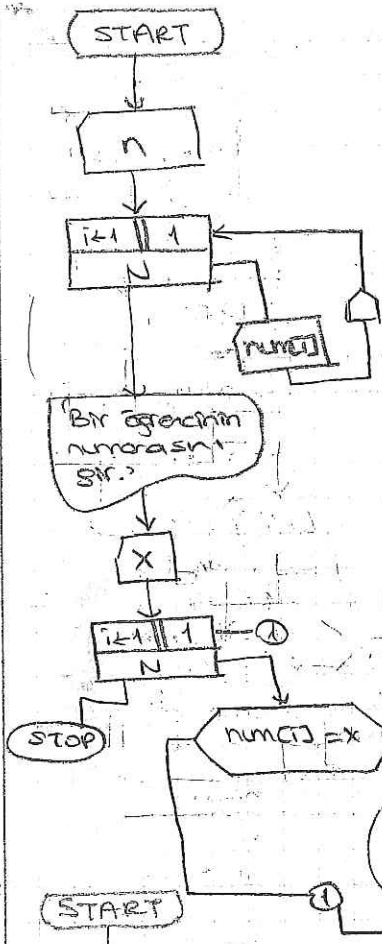




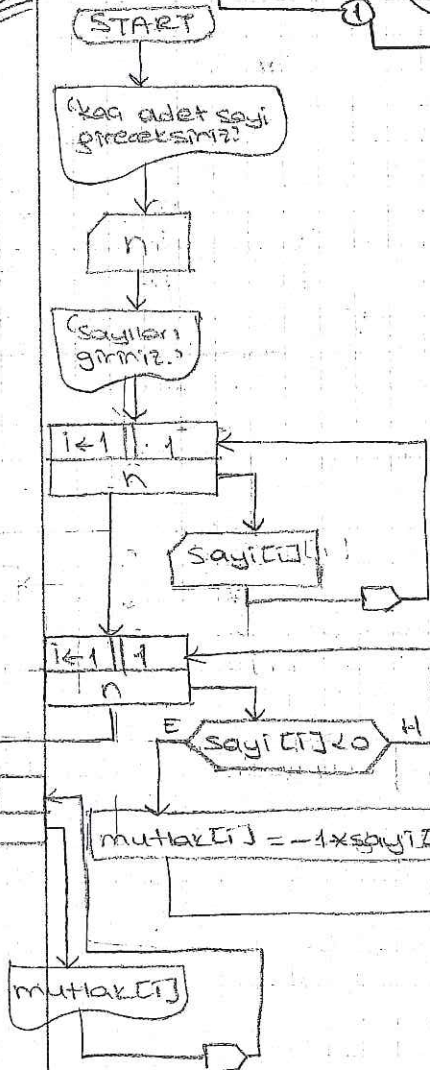
1-958

adın sayı  
belli olan  
for-

EX



EX



N=9

|   |   |    |   |    |
|---|---|----|---|----|
| 1 | 2 | 3  | 4 | 5  |
| 3 | 8 | 17 | 5 | 22 |

num x: 17

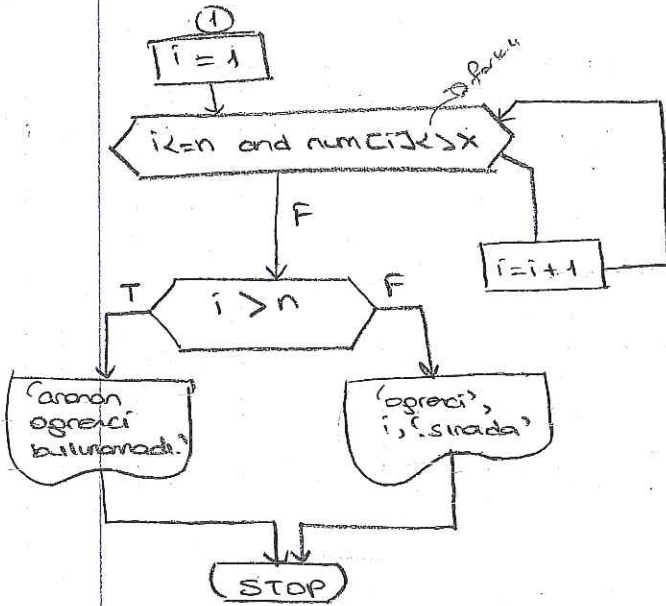
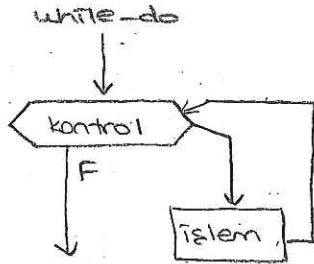
| i | num[i] | num[i] = x |
|---|--------|------------|
| 1 | 3      | 3 = 17 E   |
| 2 | 8      | 8 = 17 E   |
| 3 | 17     | 17 = 17 E  |
| 4 | 5      | 5 = 17 E   |

for i=1 to n do

Program mutlak (input, output)  
 var  
 n, i : integer;  
 sayı, mutlak: array[1..10] of integer;  
 begin  
 writeln('kaç adet sayı gireceksiniz');  
 readln(n);  
 writeln('sayıları giriniz');  
 for i:=1 to n do  
 readln(sayı[i]);  
 for i:=1 to n do  
 begin  
 if sayı[i] < 0 then  
 mutlak[i] := -1 \* sayı[i];  
 then  
 mutlak[i] := sayı[i]  
 end  
 end

for i:=1 to n do  
 writeln(mutlak[i])  
 end.

Adım sayısı belli olmayan dögüler  
 while-do : olduğu sürece  
 repeat-until : olana kadar



~~num[i] <= x and i <= n~~  
 yatarsak

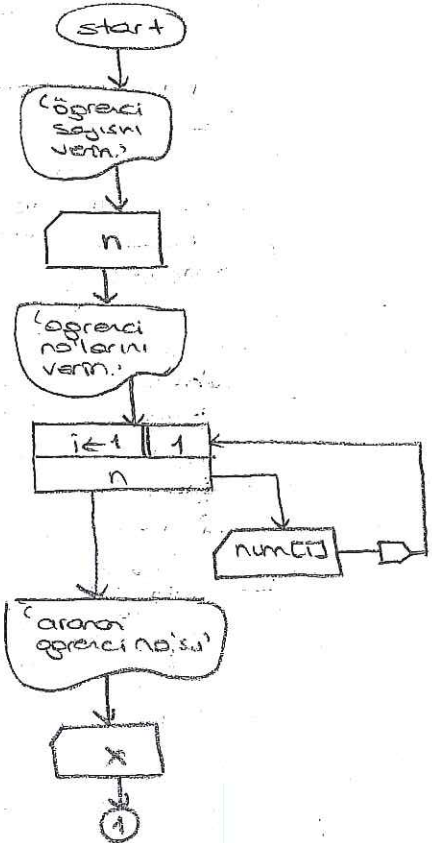
6 tane num[i] ye bakar

Çaktırdı birisi bulunmazsa  
 Her kerte bakınmaz

i <= n de num[i] ye bakarız

i < 6 de num[6] ye bakmaya çalışmaz

- ① bulunmu
- ② bulunmese ne olacak kontrol et



GENEL ANALİZ

| i | num[i] | num[i] = x |
|---|--------|------------|
| 1 | 3      | 3 < 3 → T  |
| 2 | 9      | 9 < 3 → F  |
| 3 | 17     | 17 < 3 → F |

4 nokta

1 öğrenci  
 2 öğrenci yoksa  
 ne yapar.

4 NOKTA ANALİZİ

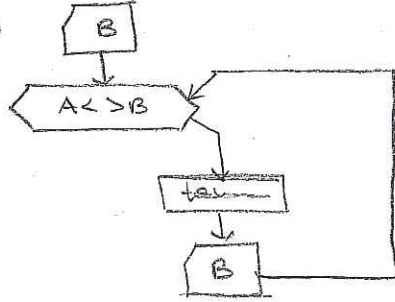
| x: 3  |    |                           |
|-------|----|---------------------------|
| 1     | 3  | 3 < 3 → T                 |
| x: 19 |    |                           |
| 1     | 3  |                           |
| 2     | 9  |                           |
| 3     | 17 |                           |
| 4     | 5  |                           |
| 5     | 22 |                           |
| 6     |    | Aranan öğrenci bulunamadı |







while →



\* bir tek  
\* repeat-until'in  
begin-endi yok

program sayi(tahmin input,output)

var

A, B, tahmin: int;

begin

writeln('1. Kizi bir sayi tutsun.');

readln(A);

tahmin := 0;

repeat

writeln('Biri yeni tahmini:');

readln(B);

tahmin := tahmin + 1

until A = B;

writeln(tahmin, 'adında dogru sayi bulundu');

end.

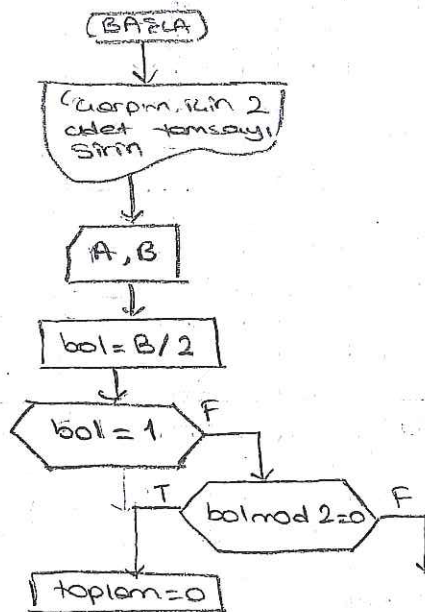
39

while algoritmasında  
başlangıç ve bitiş vardır.

İki tamsayıyı çarpmanın bir yolu sayılardan birini 2 ile çarparken diğeri 2'ye bölmektir. Başlangıçtaki sayı tek sayıysa çarpılan taraftaki sayılar toplanarak işlem bölünen sayı 1 olana kadar tekrarlanır. Toplam değer çarpım sonucunu verir. Bu yöntemle iki sayıyı çarpan algoritmayı tasarlayınız

çarp 13 26 52 104  
böl 12 6 3 1  
toplam: 0  
52  
+ 104  
13 x 12 = 156

çarp 13 26 52 104  
böl 9 4 2 1  
toplam: 0  
13  
104  
117



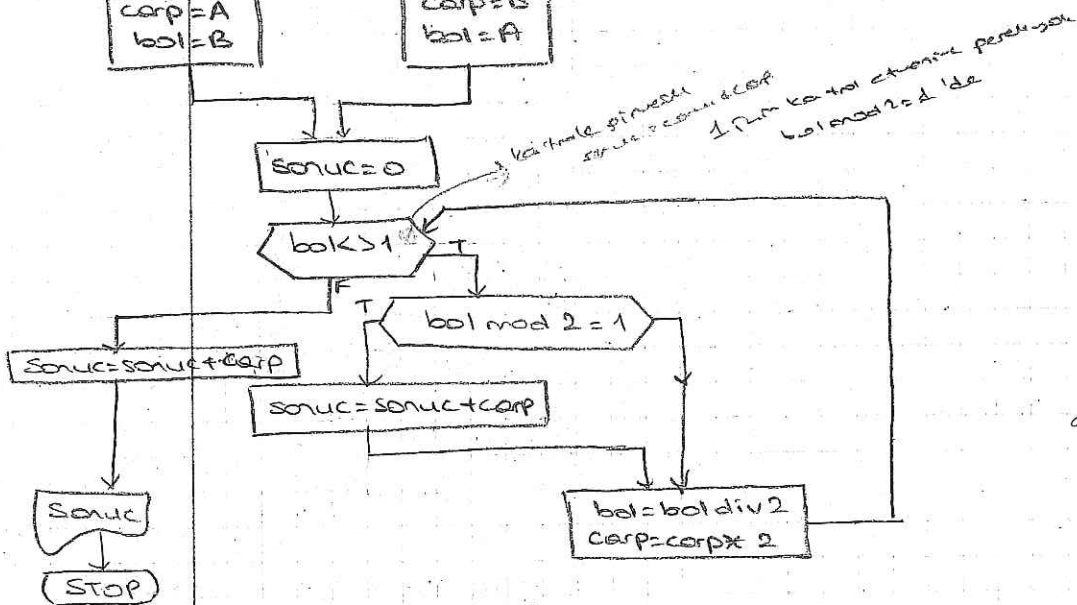
başlangıç ve bitiş  
başlangıçta olduğu  
değerler 1 ve 2  
toplanıyor

```

graph TD
    Start([Start]) --> Input[/Teki sayi veriniz./]
    Input --> Init[A, B]
    Init --> Cond{A > B}
    Cond -- T --> Box1[carp = A  
bol = B]
    Cond -- F --> Box2[carp = B  
bol = A]
    Box1 --> Join(( ))
    Box2 --> Join
    Join --> End([End])

```

| <u>carp</u> | <u>bol</u> | <u>sonuc</u>    |
|-------------|------------|-----------------|
| 13          | 9          | <del>Ø</del> 13 |
| 26          | 4          |                 |
| 52          | 2          |                 |
| 104         | 1          | 117             |



Save the best  
belonging  
and true  
reputation

Other factors  
The volume  
was not reported

|   |   |   |   |   |   |
|---|---|---|---|---|---|
|   | 1 | 2 | 3 | 4 | 5 |
| A | 3 | 1 | 4 | 7 | 5 |

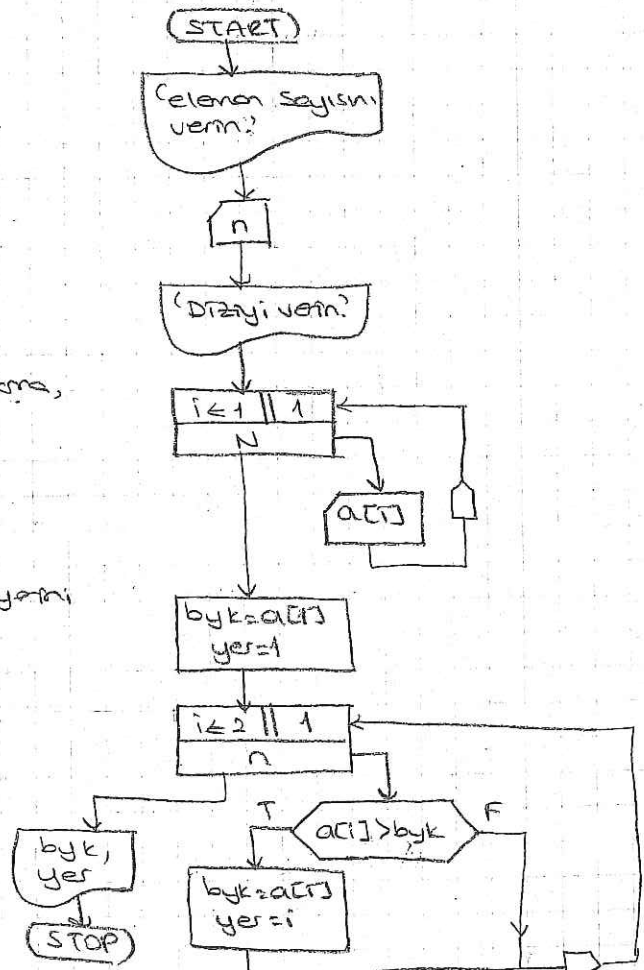
| <u>i</u> | <u>a[i]</u> | <u>byk</u> | <u>yer</u> |
|----------|-------------|------------|------------|
| 2        | 1           | 3          | 1          |
| 3        | 4           | 3          | 1          |
| 4        | 7           | 4          | 3          |
| 5        | 5           | 7          | 14         |

ama, paslarda su serbest birakma,  
nutrila, dfrin sunda olabilmek  
ei buyuk sayi verir.  
Her zaman bilinci deger olmal.

Bir dizinin 2 büyük elemanın yeri  
bulun alışı taksirleyiniz

A

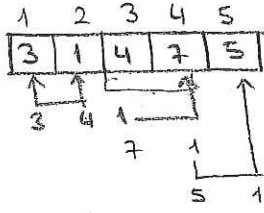
|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 1 | 4 | 7 | 5 |
|---|---|---|---|---|



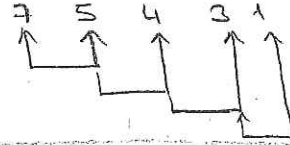


## Bubble Sort

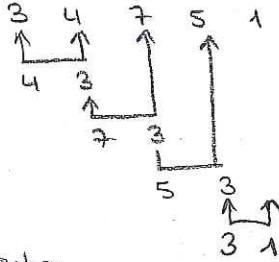
Arka arkaya 2 elemanı birbirleriyle karşılaştırıyoruz



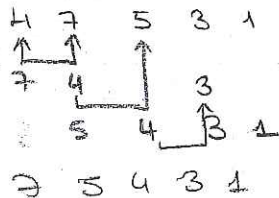
Araştırma nokta hiç bir değişiklik olmayana kadar karşılaştırma yapar.



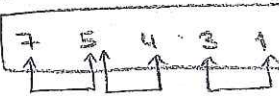
2. adım



3. adım



4. adım



yer değişmeyece kadar devam eder

## Selection Sort

A [3, 1, 4, 7, 5]

[3, 1, 4, 7, 5]

1. adım [7, 1, 4, 3, 5]

2. adım [5, 4, 3, 1]

3. adım [4]

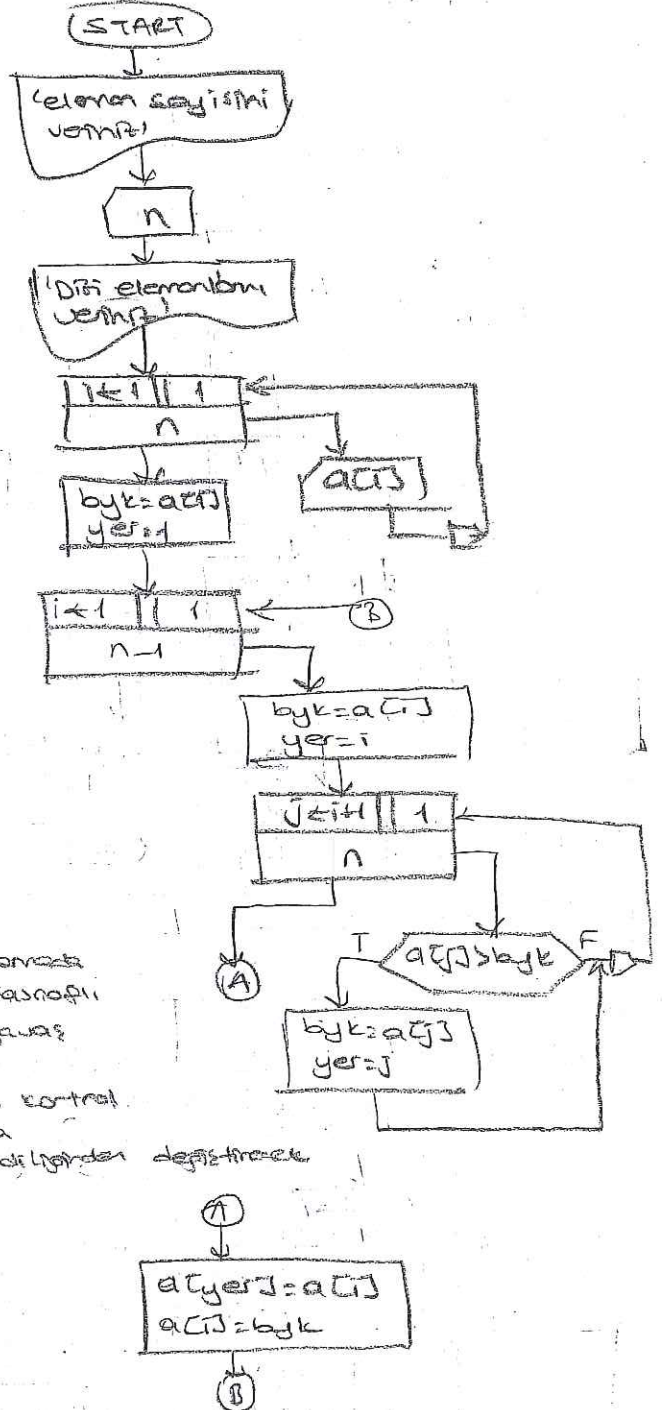
4. adım [3, 1]

Kontrol atama  
her zaman minimum  
kontrol yapar.  
Bosluğa kontrol  
yapacağına  
sayıyı kendilerden değiştirir

- 1) denekki tekrar eden ise n-1 defa
- 2) tekrar eden ise ne!

Denekki en b. elemanı bulunur ve onun  
yer değiştirilir.

Her adımda baktığım bölgenin en büyüğü  
bularak baktığım bölgenin en azından bir değere  
yer değiştiriyorum.





|    |    |   |   |   |   |
|----|----|---|---|---|---|
| 1  | 2  | 3 | 4 | 5 | 6 |
| 13 | 12 | 9 | 7 | 5 | 2 |

|    |    |   |   |
|----|----|---|---|
| 1  | 2  | 3 | 4 |
| 18 | 11 | 3 | 1 |

B

|    |    |    |    |   |   |   |   |   |    |
|----|----|----|----|---|---|---|---|---|----|
| 1  | 2  | 3  | 4  | 5 | 6 | 7 | 8 | 9 | 10 |
| 18 | 13 | 12 | 11 | 9 | 7 | 5 | 4 | 3 | 1  |

(-1) Iyi wäntem degi

(START)

 $NA, NB$ 

|    |   |   |
|----|---|---|
| 10 | 1 | 1 |
| NA |   |   |

ACia]

|                         |   |
|-------------------------|---|
| $\Gamma_b \leftarrow 1$ | 1 |
| NB                      |   |

В[Тб]

$$\begin{aligned} T_a &= 1 \\ T_b &= 1 \\ T_c &= 1 \end{aligned}$$

$T(a) = na$  and  $T(b) = nb$

$$T \leftarrow \begin{cases} \text{if } a > na \end{cases}$$

|             |   |
|-------------|---|
| $J \leq 16$ | 1 |
| nb          |   |

②

$$cT(c) = bT(j)$$

$$T_c = T_c + 1$$
$$CTCJ = a[ia]$$
$$\boxed{1a = 1a + 1}$$
$$\hat{1}e = \hat{1}e + 1$$

$A[ia] \succ B[ib]$

$$c[\pi c] = b[\pi b]$$
$$T_b = T_{b+1}$$

①

|    |   |
|----|---|
| जल | 1 |
| ह  |   |

$$C[i] = a$$

$$i = i + 1$$

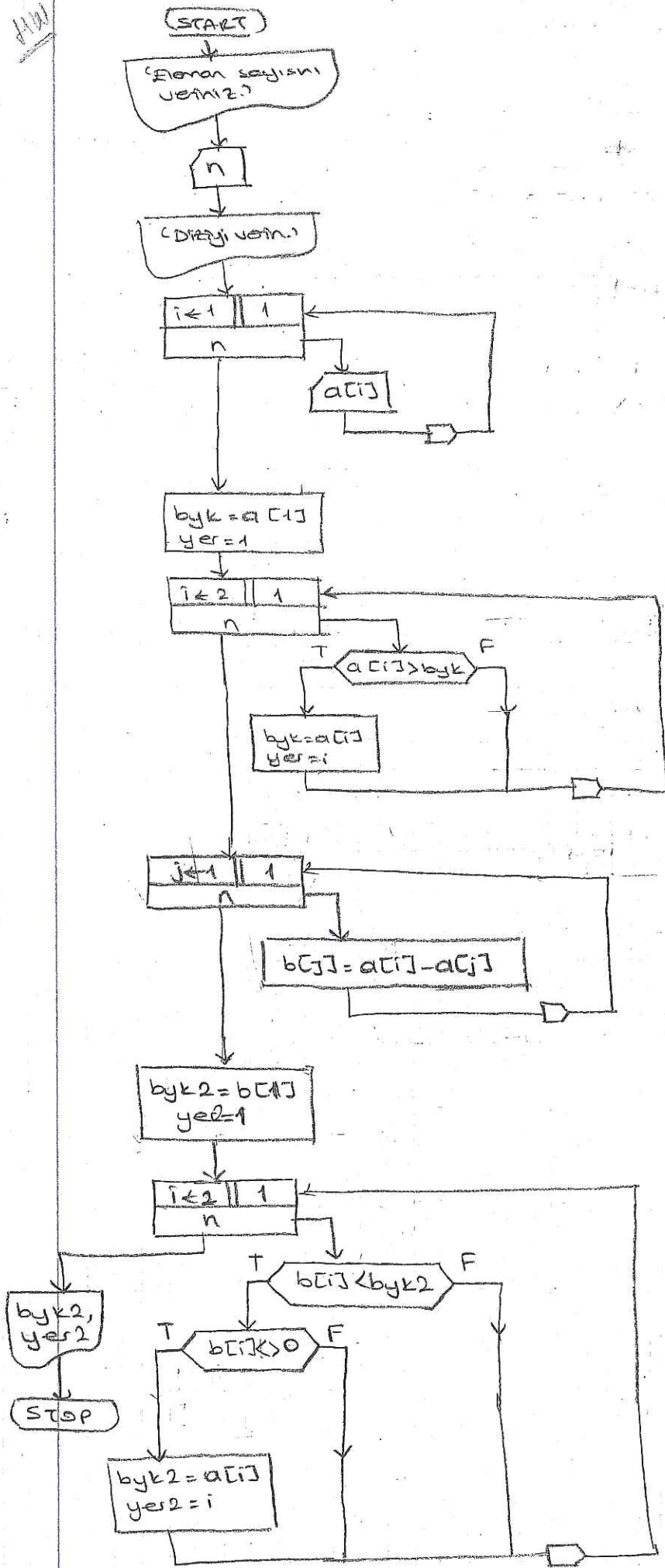
②

|                  |     |
|------------------|-----|
| $j \leftarrow 1$ | $1$ |
| nabnb            |     |

CUJJ



1/101



J 3  
 2 5  
 7 5 2 4  
 1 6

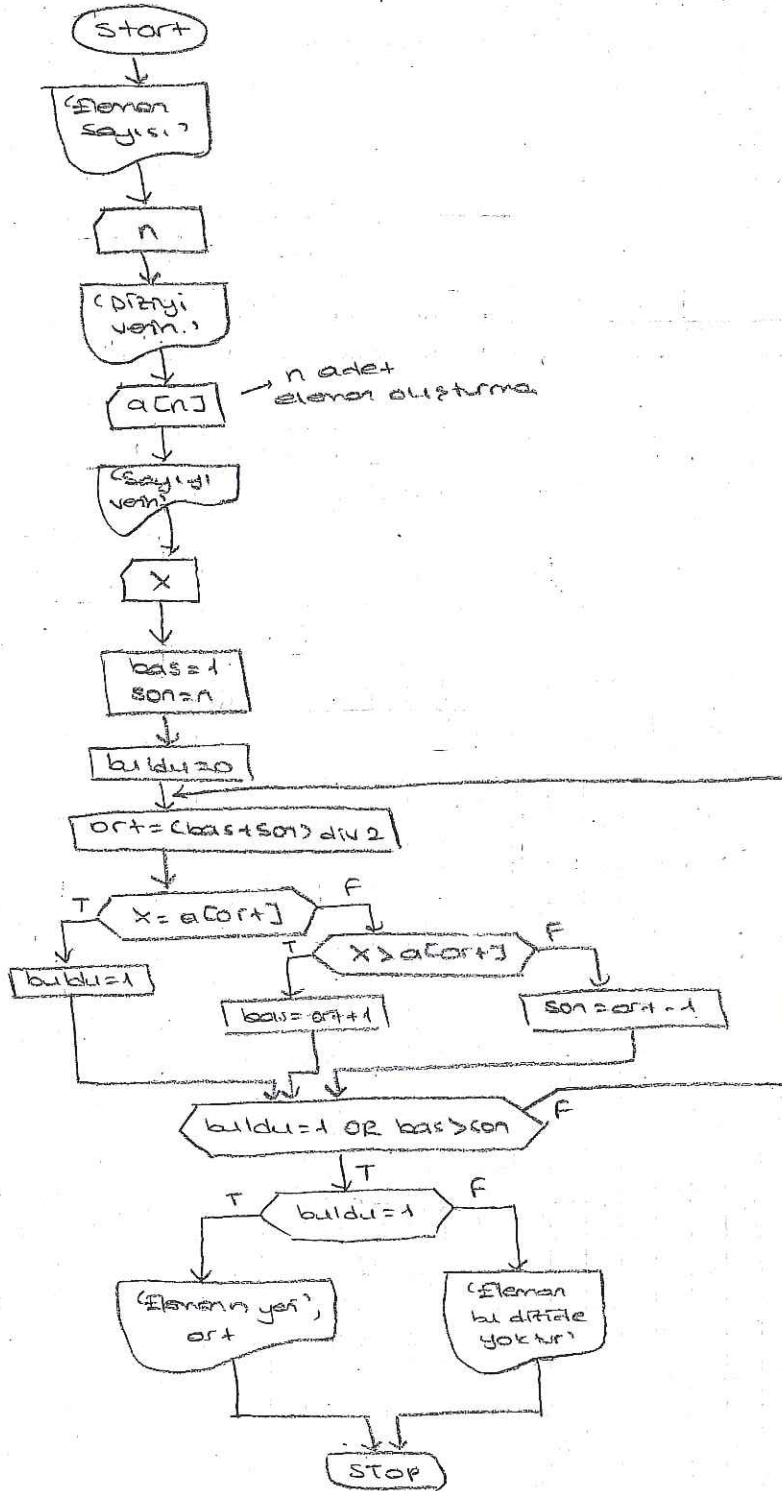






K → B

B → K'den sonra tersi



Her defa  
neden yenisiz  
kolayca  
değiştirilebilir.

buldu=1  
sonu  
değiştirilmesin

örnek 7 ile 4 - 3 sayılar kontrol  
↓  
Repeat until

örnek kontrol 4 sonra 7 ile 4  
↓  
while

i=1

| byk |
|-----|
| 2   |
| 3   |
| 4   |
| 7   |
| 7   |

| yer |
|-----|
| 1   |
| 1   |
| 3   |
| 4   |
| 4   |

| i |
|---|
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |

| a[i] > byk |
|------------|
| 1 > 3 F    |
| 4 > 3 T    |
| 7 > 4 T    |
| 7 > 7 F    |

a[yer] ← a[i]  
a[4] ← a[3]  
a[3] ← byk

i=2

| byk |
|-----|
| 1   |
| 4   |
| 5   |

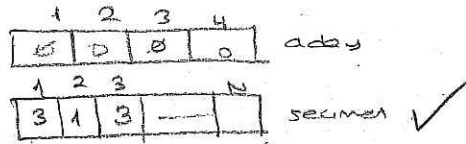
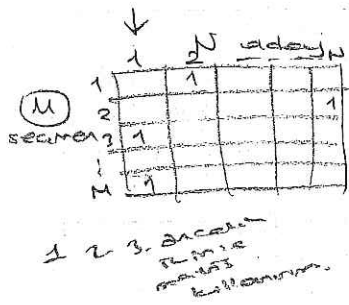
| yer |
|-----|
| 2   |
| 3   |
| 5   |

| i |
|---|
| 3 |
| 4 |
| 5 |

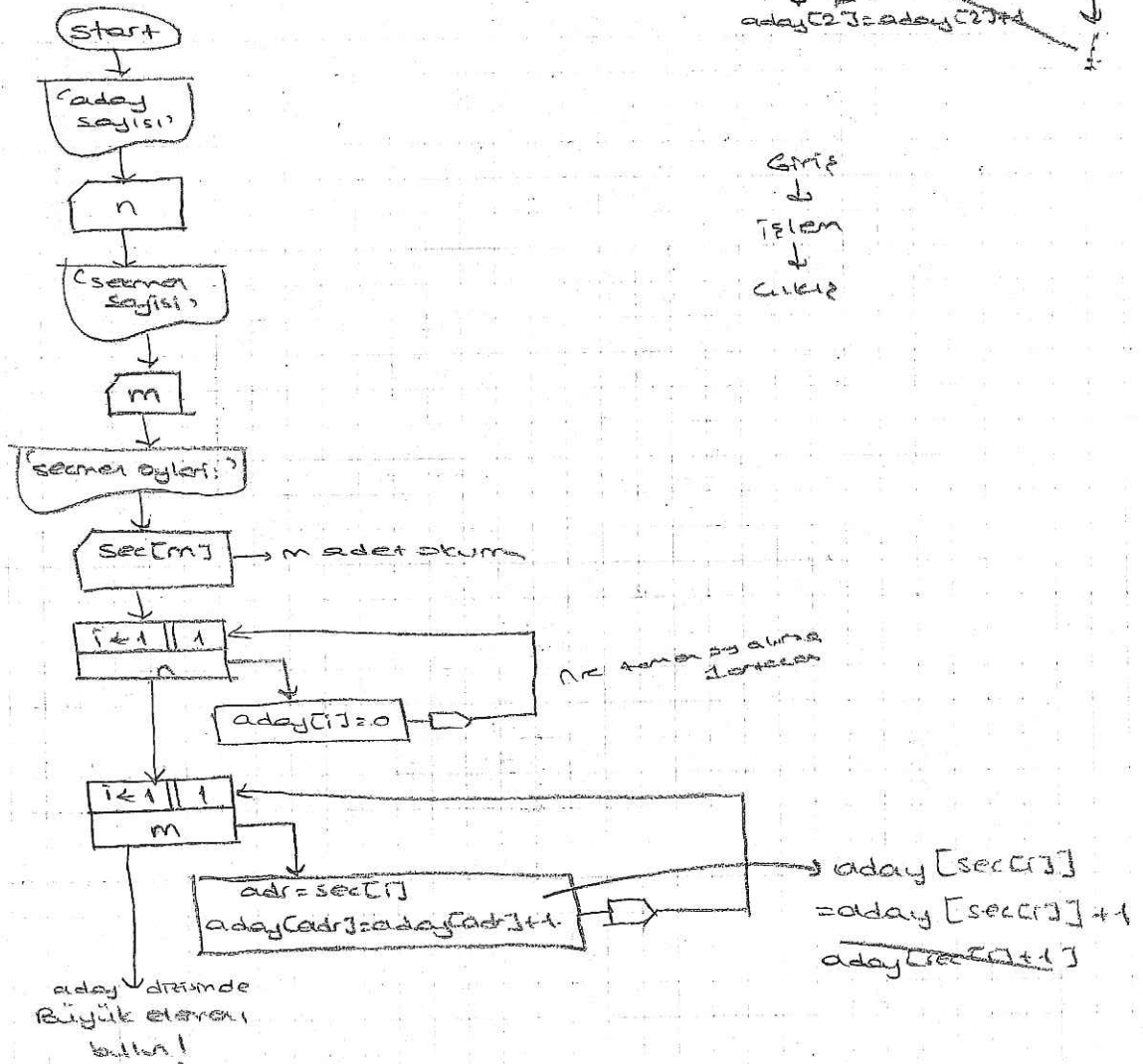
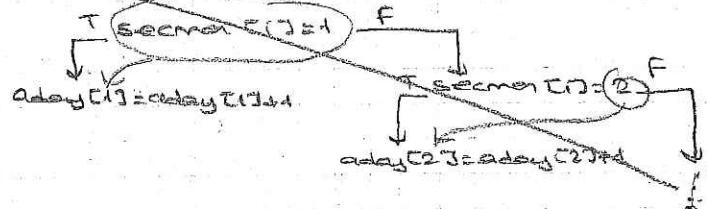
| a[i] > byk |
|------------|
| 4 > 1 T    |
| 3 > 4 F    |
| 5 > 4 T    |

a[5] ← a[3]  
a[3] ← byk

Bir maddede minter seçimi tam N adet minter adayı vardır.  
M adet seçimi de kullanılır. Her bir adayın eleme işi  
aldığı bir algoritmayı tasarlayınız.



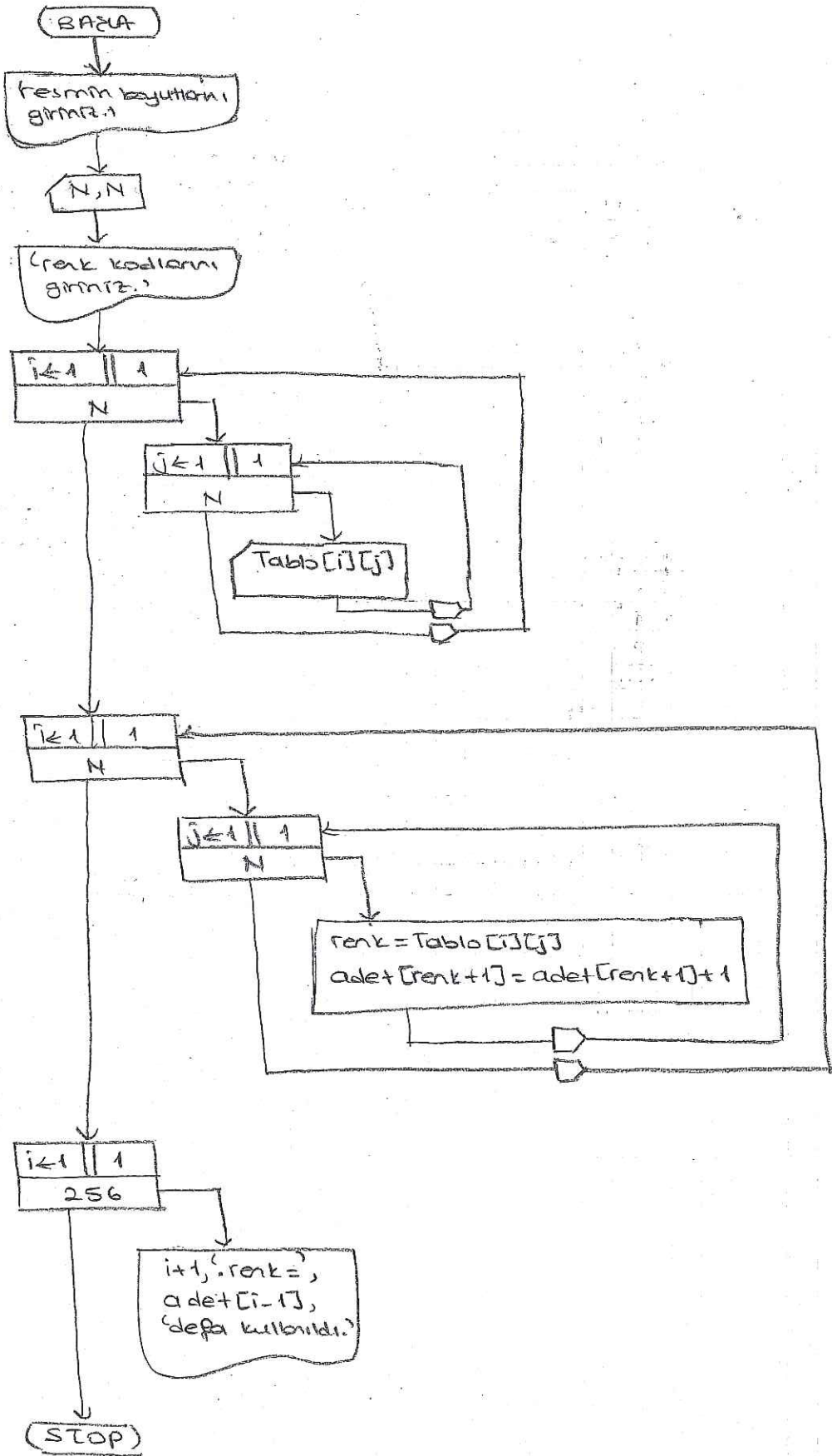
addr = secmen[i]  
aday[addr] = aday[addr] + 1



DEMS A.B.Ş.

HW

$N \times N$  lık bir resim 0-255 ye 256 renkten oluşmaktadır. Bu resimde her renk kaç defa kullanıldığını hesaplayınız





# Linear Sort with Counting

Dizide değışiklik, bilginde değışiklik  
değerli sıralama gerektir, yavaş  
ama dikkatli çalışmadan yavaş liste üzerinde

Çok zorlu kemedirince yapılmazlı olduğu ama bu özel bir durum.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 3 | 7 | 1 | 2 | 4 | 8 | 9 |

A

index dışındaki elemanların

sayısı olduğu dikkatli çalışma

bu şekilde olacağı tutulur

deneyimle ilgili olarak 7m kullanılır

K-B

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 3 | 5 | 2 | 4 | 6 | 7 |   |

9.7 kışık  
3.7. kışık

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 2 | 2 | 1 | 2 |   |   |   |
| 3 | 3 |   |   |   |   |   |
| 4 |   |   |   |   |   |   |
| 5 |   |   |   |   |   |   |

3-8 2-2  
8-3 7-1

gönye değeri  
bakmamalıym

Shavta  
kullanıcıda  
mıyda  
tutunur.

derleyici

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 3 | 7 | 1 | 2 | 4 | 8 | 9 |
|---|---|---|---|---|---|---|

A

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 2 | 2 | 1 | 2 | 2 | 2 | 2 |
| 3 | 3 |   |   | 3 | 3 | 3 |
| 4 | 4 |   |   | 4 | 4 | 4 |
| 5 | 5 |   |   | 5 | 5 | 5 |
| 6 | 6 |   |   | 6 | 6 | 6 |
| 7 | 7 |   |   | 7 | 7 | 7 |

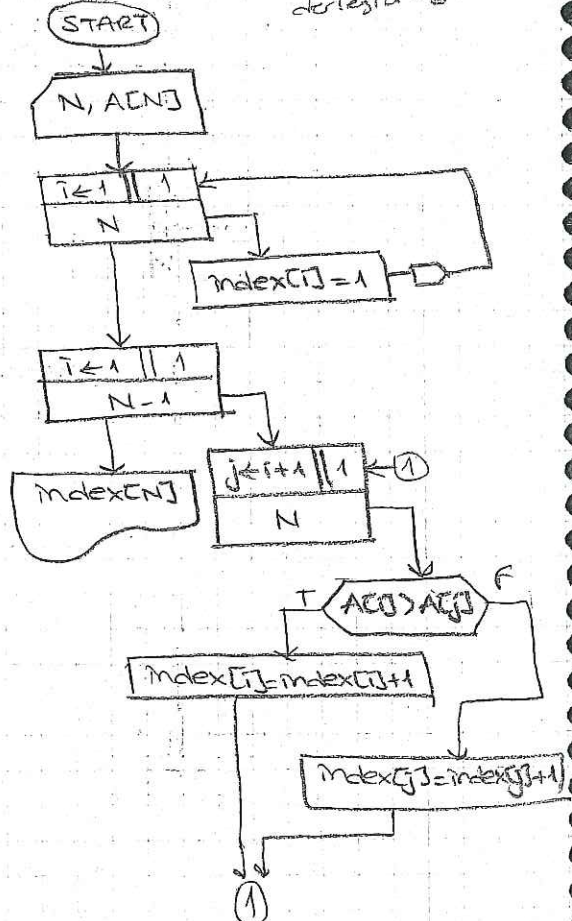
index

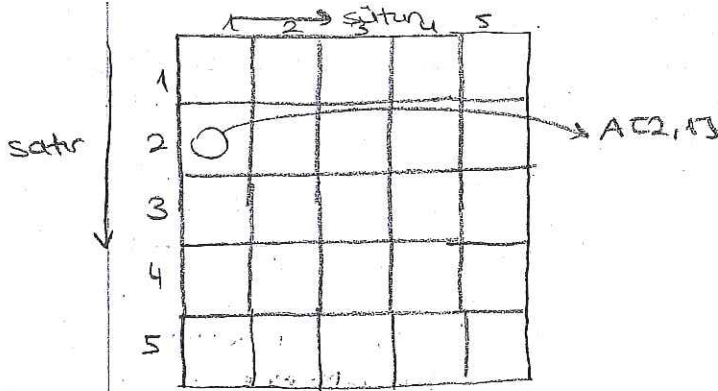
her elemanı kendinden  
sana gelenlere  
kışıkıştır  
1-1 değeri

$$i=2$$

$$C[\text{index}[i]] = A[i]$$

$$4 \quad 7$$



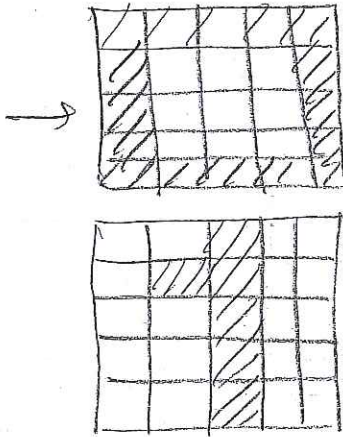


A: array[1..20, 1..10] of integer;

satr sütün

$A[M, N]$   
 $\downarrow \quad \downarrow$   
 5 4  
 satr sütün

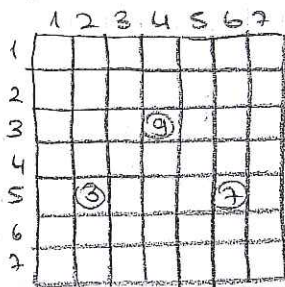
10x5x5



satır  
10 tane 5x5  
3 boyutlu bilgi

Sayı: array[1..10, 1..5, 1..5] of integer;

Sparse matrix: yarıdan fazlası 0 olan matrislere denir. Yerden kazanmak için sparse matrislerin 0 (sıfır)ları farklı elemanlar başka bir yerde saklanarak kullanılabilir. Bunun için gerekli yapıyı oluşturan algoritmayı tasarlayalım.



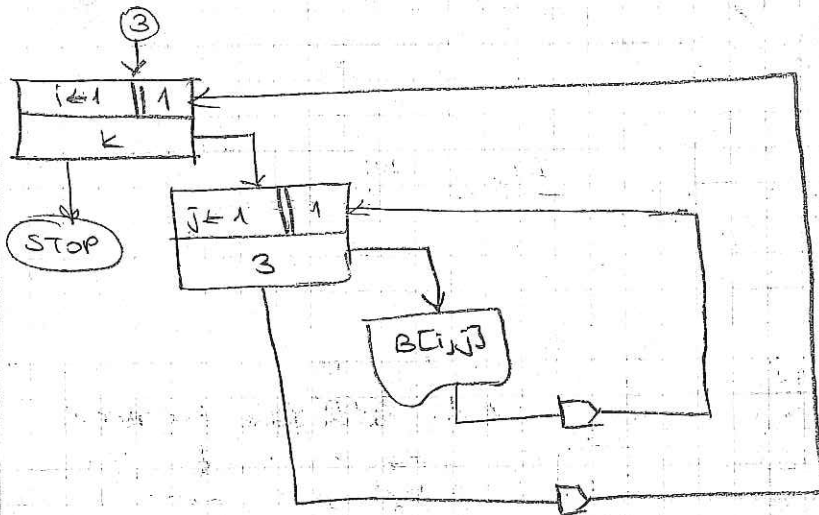
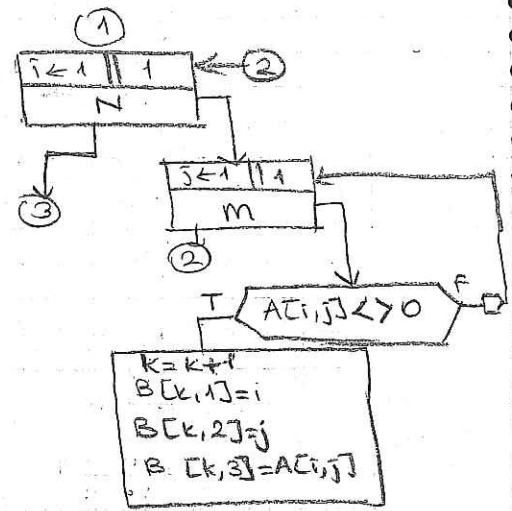
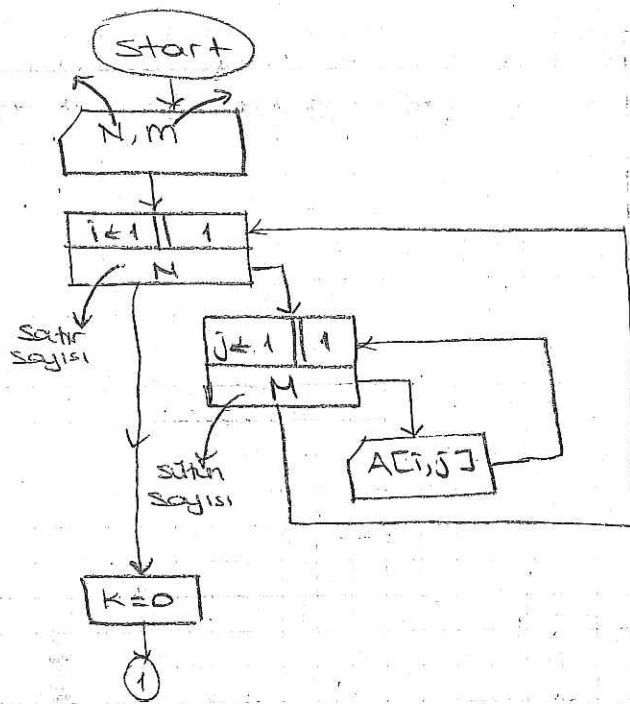
ya da birer birer bir dizi  
(yer dizi)

satır, sütün, değer

$25 \times 3 \rightarrow 75$   
 $\underline{\underline{X}}$

A: array[1..10, 1..3] of integer;





A →

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 |   |   |   |   |   |
| 2 | 4 |   |   |   |   |
| 3 |   |   |   |   | 9 |
| 4 |   |   |   |   |   |

→ k=1  
k=2

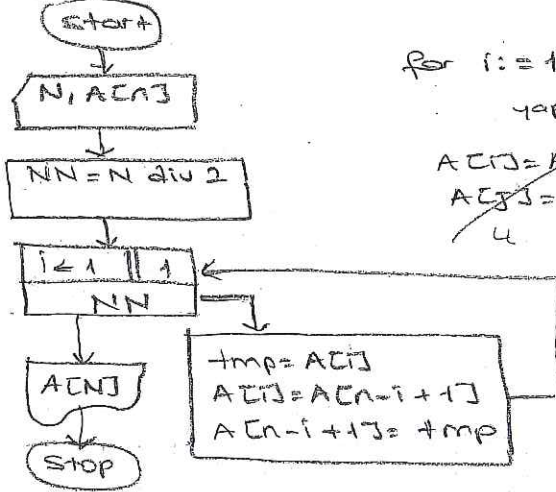
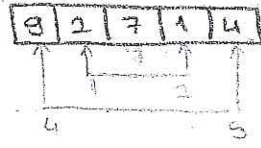
$A[3,5] \neq 0$

| 1     | 2     | 3     |
|-------|-------|-------|
| Sutun | Sutun | deger |
| 2     | 1     | 4     |
| 3     | 5     | 3     |

var  
b: array [1..40, 1..30] of integer;  
a: array [1..100, 1..100] of integer;  
 $a[i,j] := 9;$



Bir diziin elemanların 1. ve n. sırası, 2. ve n-1. sırası, 3. ve n-2. sıradaki elemanları yerini değiştiririz. Tasarlayalım.



for i = 1 to  $\lfloor N \div 2 \rfloor$   
yapma  
 ~~$A[i] = A[j]$~~   
 ~~$A[j] = A[i]$~~   
~~4 4~~

NN=2

| i | N-i+1 | tmp | A[i] | A[N-i+1] |
|---|-------|-----|------|----------|
| 1 | 5     | 9   | 4    | 9        |
| 2 | 4     | 2   | 1    | 2        |

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 3 | 7 | 0 | 0 | 0 |
| 2 | 4 | 0 | 0 | 0 | 0 |
| 3 | 1 | 5 | 9 | 0 | 0 |
| 4 | 2 | 1 | 8 | 4 | 0 |
| 5 | 3 | 0 | 0 | 0 | 0 |

1. satır 4. satır  
2. satır 3. satır  
3. satır 1. satır  
4. satır

Adres

| 1 | 2 | 3 | 4 | 5  |
|---|---|---|---|----|
| 1 | 3 | 4 | 7 | 11 |

Değer

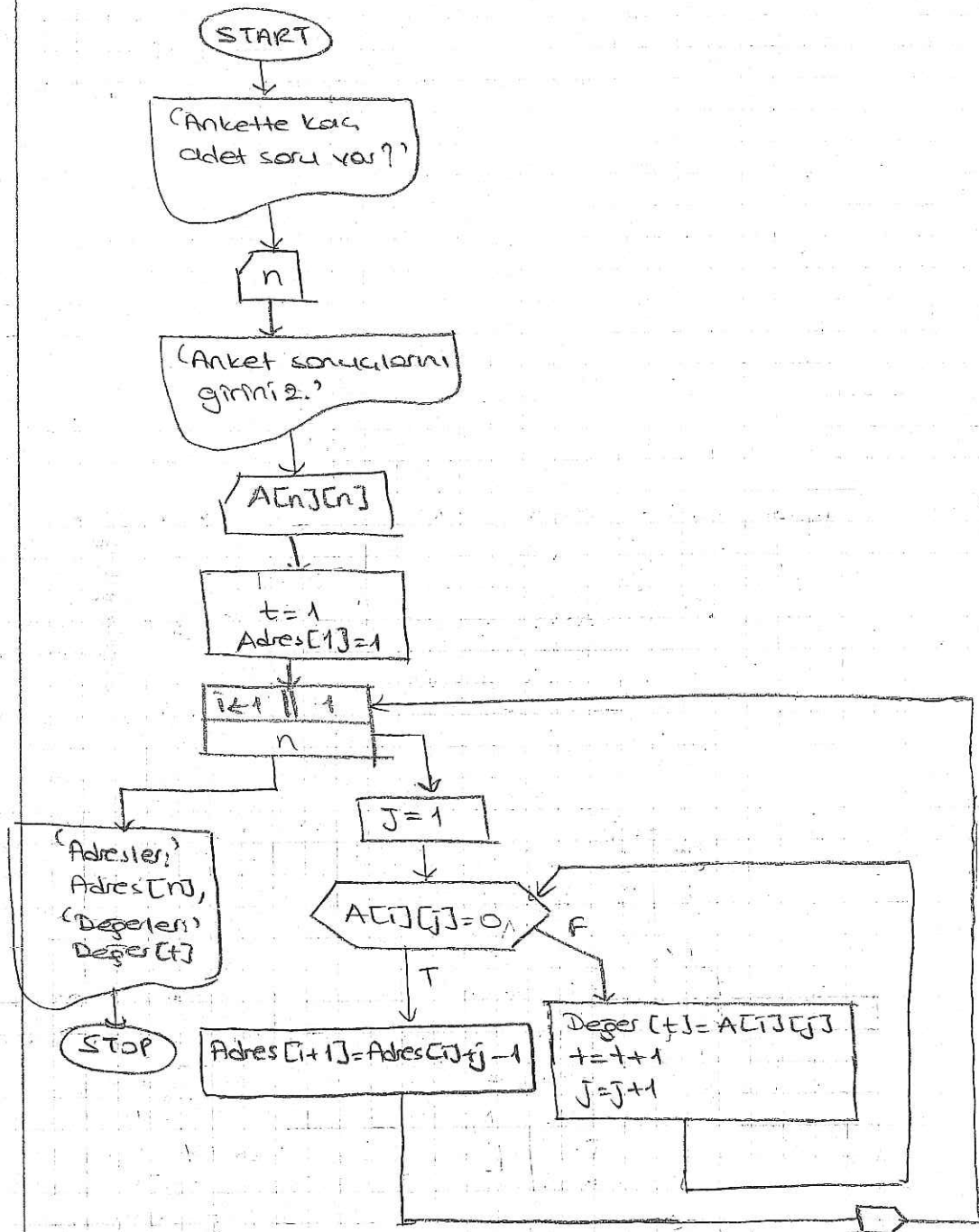
|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 3 | 7 | 4 | 1 | 5 | 9 | 2 | 1 | — |
|---|---|---|---|---|---|---|---|---|

1. satır 2. satır 3.

Bir anket sonunda n sorunun en fazla n  
sıklıkla vardır. Buna göre at sıklık sorular  
tam 0 sorunun bulunduğu satırda  
sıklıkla bir tane yerden sonra sıklıkla 0 olur  
N de doldurulmuştur. Yeterden kazanmak için bu  
matrisin 0'ları ile farklı elemanları  
bir diziye aşağıdaki yapıda yerleştiririz  
algoritmayı tasarlayalım.

Her satır ne kadar boşluk  
bulunuyorsa

uygun yapıyı oluşturduktan sonra  
A matrisinin i'ye j. elemanının  
değerini  $A[i][j]$  yer oluşturduğumuz  
yapıyı kullanarak buluruz



### ANALİZ

| <u>n</u> | <u>i</u> | <u>j</u> | <u>t</u> | <u>Adres[i+1]</u> | <u>Değer[t]</u> |
|----------|----------|----------|----------|-------------------|-----------------|
| 5        | 1        | 1        | 1        | 1                 |                 |
|          |          | 2        | 2        |                   | 3               |
|          |          | 3        | 3        | 3                 | 7               |
|          | 2        | 1        |          |                   | 4               |
|          |          | 2        | 4        | 4                 |                 |
| 3        | 1        | 1        |          |                   | 1               |
|          |          | 2        | 5        |                   | 5               |
|          |          | 3        | 6        |                   | 9               |
|          | 4        | 1        | 7        |                   | 2               |
|          |          | 2        | 8        |                   |                 |

# Sieve (Kalbur) Yöntemi ile Asal Sayı Bulma

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 ... N

Eleme yöntemi, yedek dizi doğruya geçek yok

asal

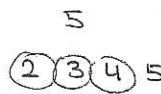
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 ... 1

3 →

5 →

7 →

Karşıma gelene kadar zaten katlarına örnekler içinde ugramı



$\sqrt{N}$ 'e kadar gidecek

$\sqrt{N}+1$

7 → 49 yok

11 → 121 yok

eleymezim.

karşıma gelene kadar.

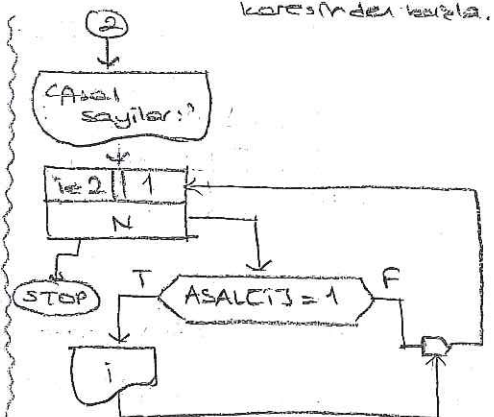
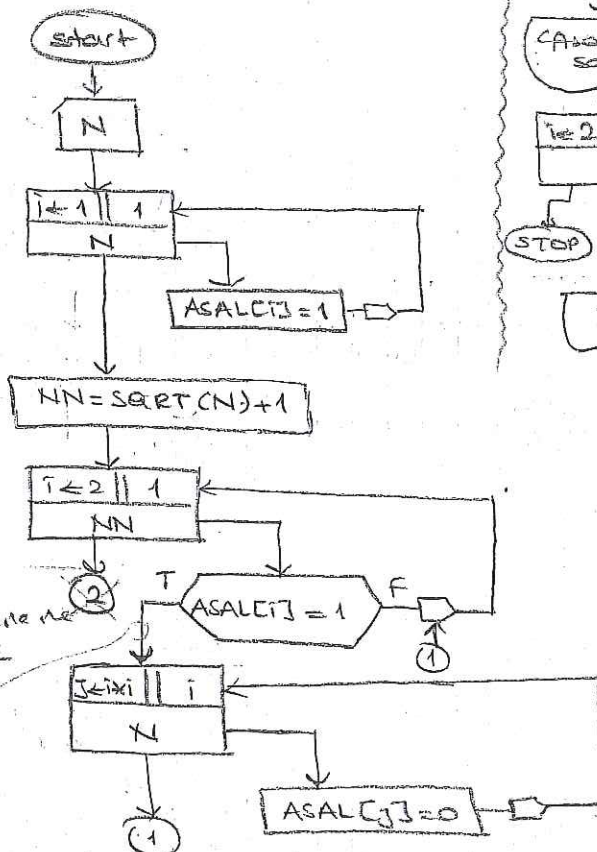
karşıma gelene kadar  
x'in  
katları  
elecek

Write

poncaldaki write ne  
(dışarı  
olduğuna)

asal  
sayılar  
buluyoruz

karşıma gelene kadar





**BÖNEM  
ÖDEVİ**

market kasası simülasyonu.  
(kuyruk)

K1 K2 K3 K4

Bir markette 4 tane kasa var.  
Başlangıçta hiçbir kaside müşteri yok.  
Bir random sayı üretili kullanılarak her adımda önce kasalara döne  
gelen müşteri olup olmadığı kontrol edilir. Eğer müşteri varsa yine  
random olarak <sup>kuyruktaki</sup> işin ne kadar süreceği belirlenir.  
(Çapattaki esya da random belirlenir)  
Her müşteri en kısa kuyruğa yerleştirilir ve aynı zaman  
dönümde kuyruğun en sonunda müşterinin kuyruktaki zamanı 1  
arttırılarak sırası olan müşteri kuyruktan çıkartılır, aynı  
yerine arkasındaki müşteri gelir. Bu simülasyonu yapan  
sistemi tasarlayınız.

Örnekte  
her adımda

random

$X \% = \text{random}(10)$

Sıra num.

$0 \rightarrow N-1$   
 $1 \rightarrow N-1$

1 ve 9 arası sayı üretilir.

şu kassaya gideceğine işine karar verilir.

Sekizinci

1 önce müşteri var mı yoksa sadece kuyruğun en sonundakı

2 işi ne kadar sürer 5

Sadece en sonundakı  
dışta

ek kassalardan biri kullanılır

K1 K2 K3 K4  
~~0~~ ~~1~~ ~~2~~ ~~4~~ ~~5~~ 1 ~~2~~ ~~0~~ ~~4~~ ~~5~~  
Müşteri var mı yok mu

1 adet yerleştirme

(her kuyruktakinin işi 1 arttırdı)

kuyruktakilerin sırası  
beklerken ilerliyor.

Uygulanır iş yapma süreci (simülasyon)

ekranı görür → her adım sonunda  
nereye gideceğine

(pozitive formata  
yazar)

kuyruğun başı

eterele basılır.

→ sonrakı adım  
müsteri kasa yerleştirilir, gideceğine

t anında

→ K1 K2 K3 K4  
4 3 5 9  
9 5 4 1  
1

Sıra olarak  
en kısa

eski kasa  
gözetilir.

random 11 müşteri sayısı 10 olan

0-1000 mod 2 yapar  $X \% = \text{random}(1000) \text{ mod } 2$

deneyim 2, der 0 (kuyruk zamanı 0 olarak)

So-4

üstteki ifadeye de bir işleme (örneğin)

→ eğer işi yapar at.

bu karte de her 1 parçayı kuyruğa girer, sonra işlem yapılır

→ Queue simülasyonu

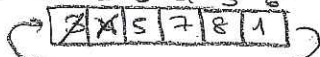
FIFO  
First in First out



X 9 4

LIFO  
Last in First out

baş, son  
baş ↓ son  
1 2 3 4 5 6  
son



Circular queue  
(dairesel kuyruk)

Kapalı bir dairesi gibi

effective bir çözüm değil.

4 kuyruktan çıktı

baş = 3

son = 1

mod 2 başa  
dönmesi lazım

→ eğerde bütün işi bitireceğiz.

X := A + B \* C - D

stack kullandılar  
kuyruk baş son

|               |   |   |
|---------------|---|---|
| 3             | 1 | 1 |
| 3, 4          | 1 | 2 |
| 3, 4, 5       | 1 | 3 |
| 3Cn istedi    |   |   |
| 4, 5          | 2 | 3 |
| 4, 5, 8       | 2 | 5 |
| 4, 5, 8, 1    | 2 | 6 |
| 4, 5, 8, 1, 9 | 2 | 1 |

(bunu yaparken 1 göre serbest bırakıyoruz)

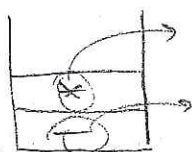
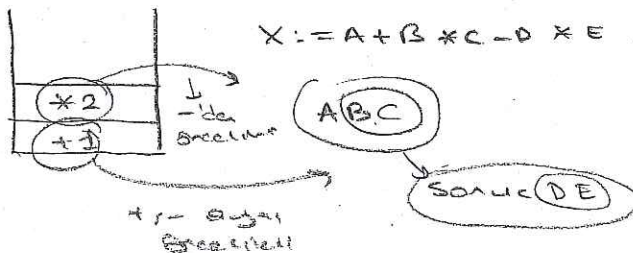
baş 2 de 1 serbest

5, 10  
→ 10 kadar

\* Her kuyruk en fazla n kisi almaktadır. Bunu da kuyruk dolmuş  
o kuyruk kize bile olsa üstteki alınmamaktadır.

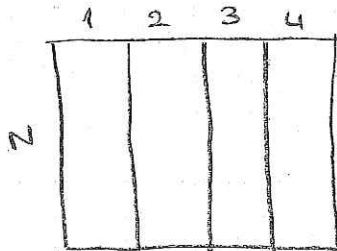
(Alınmayan 2 kaza?)

↓ bir zamanlar  
→ kaza oldu, parçaları  
geldiye göre  
→ kize ya da ekle, 2n içinde 1 çıktı



Bacaktığızda sonuç geldiye kadar bekletiyor.





Tesim tarihi final günü C15 özetli  
pzt.

tesim son günü

ÖDEV RAPORU şeklinde  
%20

yaptığın her çalışmaya değeri

1 sayfa → her problem tarihi  
çalışılır şekilde  
yaptığın işin tarihi

→ arkadaşlar blok

diagramlar 1 denon

ünlüde yapılı rı.

(genel akis diagramı)

(kuyruğu nasıl çalışır,

en kısa kuyruğu nasıl bulur)

örnek kuyruğu 1 denon

yer göre en kısa çözüm

↑  
nisi bulma  
merak etmen

genel akis diagramı

551e yaptığın rı

gösterir.

Özel detaylı akis diagramı

detaylı veriyorsun

sonra da

parçaları da ki cote

Conrad cetrüyle

değışken cetrüde

// değışkenlerin yalınca

calculation yapt.

deklare ederken

Arkadaşlar her 1000'e yaptığınız iş anıyorsunuz.

başlıca -

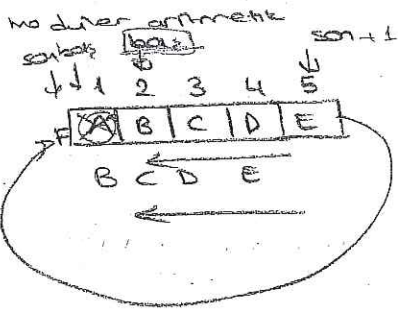
her işin başlangıcı rı.

Programa bakıldığında program anlaşılır olmalı

(Kuyruğu kuyruğu yapısında istenirse 25 rı)



Öğrenci, aşağıdaki görevleri yapar.



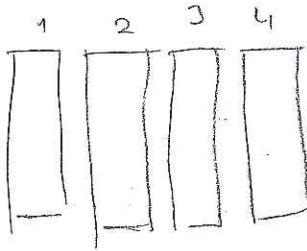
modüler aritmetik

$son = son + 1$  5 ile 6 alınarak  
mod 5

random(5)  
0-4

baş = 0  
son = 1

baş = 0  
son = 0



baş, son dizisi

if 3x

4 değeri 40 kaçan oldu?

if yanıtı yanlış.  
and → 2 if

4 kere tüm dizi, baş kontrolü

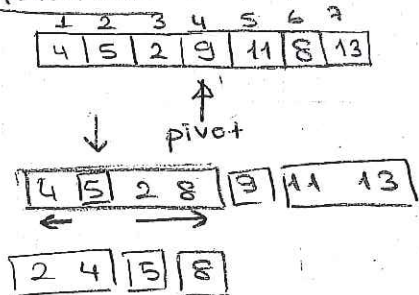
Bir matriste her satır ve sütun kendi içinde sıralıdır. Dizilerde verilen bir elemanı yemi a uygun şekilde bulma algoritmasını tasarlayınız.

|    |    |    |    |    |    |  |
|----|----|----|----|----|----|--|
|    | 11 | 12 |    |    |    |  |
| 11 | 1  | 4  | 7  | 9  | 13 |  |
| 21 | 3  | 11 | 17 | 19 | 21 |  |
| 31 | 10 | 14 | 24 | 27 | 28 |  |
|    | 13 | 16 | 31 | 33 | 39 |  |
|    | 51 | 55 | 59 | 69 | 72 |  |

X = 35

Ya bastır ya sende basla

Quick Sort



Geçerli Adım

Pivotun soldakilerden daha küçük, pivot, daha büyük sağ  
sagdakilerden daha büyük

rekürsif

recursive

1) 7 el.

2) 4 el.

3) 2 el.

hersekteki adı parçaların  
baş ve son değeri

n elemanlı bir dizinin  $\frac{n}{2}$  elemanını pivot kabul ederek

pivotlar küçük elemanlar dizinin baş tarafına, büyük elemanlar da  
sağ tarafına yerleştirilerek algoritmayı tasarlayınız. (Elemanların  
sıralı yerleştirilmesi zorunda değil)

|    |   |   |    |   |    |    |   |   |    |    |    |    |
|----|---|---|----|---|----|----|---|---|----|----|----|----|
| 1  | 2 | 3 | 4  | 5 | 6  | 7  | 8 | 9 | 10 | 11 | 12 | 13 |
| 11 | 5 | 9 | 13 | 8 | 17 | 10 | 2 | 4 | 13 | 16 | 7  | 14 |

$\uparrow$  (at index 1)  
 $\uparrow$  (at index 4)  
 $\uparrow$  (at index 5)  
 $\uparrow$  (at index 7)  
 $\uparrow$  (at index 12)

|    |   |   |    |   |    |    |   |   |    |    |    |    |
|----|---|---|----|---|----|----|---|---|----|----|----|----|
| 1  | 2 | 3 | 4  | 5 | 6  | 7  | 8 | 9 | 10 | 11 | 12 | 13 |
| 11 | 5 | 9 | 13 | 8 | 17 | 10 | 2 | 4 | 13 | 16 | 7  | 14 |

10 → 9 → 7 → 15 → 11 → 13  
 7 → 8 → 10

15 < 10

adr  
2

i  
2

3

3

→ 9 ↔ 15

4

4

5 8 ↔ 15

6

7

8

9

10

11

5

12

13

7 ↔ 13

kontrolasi data apakah bkr atau m.

di sini buldgan kuantitas

photon

yaofit degnitiralin

Start

N=7

N, ACN

orta = (1+N) div 2

pivot = a[orta]

a[orta] = a[1]  
a[1] = pivot

adr = 1

i < 2 || i > N

a[i] = a[adr]  
a[adr] = pivot

acn  
STOP

adr = adr + 1

tmp = a[i]  
a[i] = a[adr]  
a[adr] = tmp

|    |   |    |   |    |   |   |
|----|---|----|---|----|---|---|
| 12 | 1 | 17 | 8 | 22 | 3 | 9 |
|----|---|----|---|----|---|---|

adr  
1

i  
2

degnitiralin

2

1 ↔ 1

3

4

5

6

3

17 ↔ 3

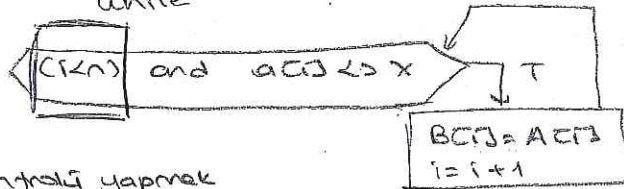
7

|    |   |    |    |    |    |   |
|----|---|----|----|----|----|---|
| 12 | 1 | 17 | 8  | 22 | 3  | 9 |
| 8  | 3 | 8  | 12 | 17 | 22 | 9 |

12 22 17 9  
 12 3 17 22

2

while



kontrolü yapmak  
sonuçları.